

ZYLO •

ENGINEERING DOCUMENTATION

zylo

Live Streaming & Creator Platform

Architecture, APIs, real-time protocol, data model, security, testing and operations, consolidated from fourteen engineering documents into a single reference.

COMPILED

24 September 2026

CONTENTS

14 documents • 6 parts

Next.js • Express • PostgreSQL • Redis • Socket.IO • LiveKit • Razorpay

Contents

PART I — OVERVIEW

01 Platform Overview	4
1.1 Key Features	4
1.2 Technology Stack & Infrastructure	5
1.3 Repository Directory Structure	5
1.4 Prerequisites	6
1.5 Local & Production Development Setup	6
1.6 Testing	6
1.7 Documentation Suite	7
02 Repository & Project Structure	8
2.1 Root Repository Structure	8
2.2 Backend Module & File Responsibilities (/backend)	8
2.3 Frontend Directory & File Responsibilities (/client)	10

PART II — ARCHITECTURE

03 System Architecture	11
3.1 Current Implementation Architecture	11
3.2 Request Lifecycle & Layer Breakdown	12
3.3 Real-Time & Live Streaming Architecture	12
3.4 Virtual Wallet & Gift Transaction Architecture	13
3.5 Master Architecture Diagrams	15
3.6 Recommended Production Architecture Roadmap	17
04 Architectural Decision Records	18
ADR 001 Selection of Neon PostgreSQL & Prisma ORM	18
ADR 002 Managed LiveKit Cloud for WebRTC Live Streaming & Egress Recording	19
ADR 003 Frontend Deployment on Vercel & Backend Hosting on AWS EC2 + Nginx	19
ADR 004 Socket.IO for Real-Time Chat & State Synchronisation	20
ADR 005 Dual-Balance Virtual Wallet with Row-Level FOR UPDATE Locking	20
ADR 006 Argon2id Password Hashing & Refresh Token Family Rotation	20
05 End-to-End Data Flows	21
Flow 1 User Registration	21
Flow 2 User Login	22
Flow 3 Start Live Stream (Creator)	22
Flow 4 Viewer Joins Live Stream	23
Flow 5 Send Real-Time Chat Message	23
Flow 6 Send Virtual Gift	24
Flow 7 Razorpay Wallet Top-Up & Verification	25
Flow 8 Stream End & Recording Finalization	25

PART III — INTERFACES

06 REST API Specification	26
6.1 Master API Endpoint Inventory	27
6.2 Endpoint Deep-Dives & Schemas	29
07 API, Data & Caching Architecture	32
7.1 Executive Summary & Core Principles	32
7.2 Data Classification System	32
7.3 Complete API Inventory & DTO Specifications	33

7.4	Redis Key Inventory	36
7.5	Realtime Socket.IO Event Map	37
7.6	Cache Invalidation Matrix	37
7.7	Performance & Query Optimization Strategy	37
08	Real-Time Protocol (Socket.IO)	38
8.1	Connection & Authentication Handshake	38
8.2	Room Architecture	38
8.3	Master WebSocket Event Inventory	39
8.4	Chat Rate Limiting & Validation	40
8.5	Reconnection & Cleanup Lifecycle	40
PART IV — DATA		
09	Database & Schema	41
9.1	Relational Database Schema Table Inventory	42
9.2	Table Column Specifications & Data Types	42
9.3	Database Concurrency & Row-Level Locking Analysis	43
9.4	Indexing & Query Efficiency	44
PART V — INFRASTRUCTURE & OPERATIONS		
10	Deployment & DevOps	45
10.1	Production Architecture Overview	45
10.2	Component Deployment Guides	46
10.3	Production Environment Variables Specification	50
10.4	Production Health Check & Monitoring	50
11	Operational Runbook	52
11.1	Daily Operations & Service Commands	52
11.2	Database Maintenance Commands	53
11.3	Redis Maintenance & Cache Management Commands	53
11.4	Log Inspection & Troubleshooting Runbook	53
11.5	Common Operational Failures & Resolution Steps	54
PART VI — QUALITY & SECURITY		
12	Security Audit & Threat Model	55
12.1	Authentication & Token Hardening	55
12.2	Role-Based Access Control (RBAC) Matrix	56
12.3	Forensic Security Vulnerability Audit	56
13	Automated Testing Audit	58
13.1	Test Framework Setup	58
13.2	Existing Test Inventory	58
13.3	Assessment Requirement Compliance Audit	59
13.4	Recommended Additional Automated Tests	60
14	Technical Audit & Compliance	61
14.1	Executive Technical & Infrastructure Summary	61
14.2	Requirement Compliance Matrix	61
14.3	Critical Findings & Technical Debt Inventory	63
14.4	Production Readiness Roadmap	63

01

Platform Overview

Zylo is a modern, high-performance live streaming platform built with Next.js 13, Express, TypeScript, LiveKit Cloud WebRTC, Socket.IO, Neon PostgreSQL (Prisma), Redis, Razorpay, and Cloudflare R2 / AWS S3 storage.

1.1 Key Features

- **Live Streaming & Realtime WebRTC:** Broadcaster studio powered by LiveKit Cloud for low-latency sub-second streaming and high-definition video publishing.
- **Production Infrastructure:**

FRONTEND	Next.js App Router deployed on Vercel .
BACKEND	Express API & Socket.IO server hosted on AWS EC2 (Ubuntu 26.04 LTS) behind an Nginx reverse proxy with SSL.
DATABASE	Serverless Neon PostgreSQL with Prisma ORM.
MEDIA STORAGE	Cloudflare R2 / AWS S3 for stream recordings and Cloudinary for image asset uploads.

- **Realtime Chat & Moderation:** Low-latency Socket.IO chat rooms with live viewer counting, chat rate limiting, mutes, temporary bans, and permanent bans.
- **Virtual Gift Economy & Dual-Wallet System:** Atomic coin transactions supporting both **Purchased Coins** and **Creator Earnings** with row-level database locking (**SELECT ... FOR UPDATE**), idempotency keys, and real-time gift animations.
- **Razorpay Top-Up Integration:** Automated INR-to-coin top-up order generation and HMAC SHA256 signature verification.
- **Social Graph & Discovery:** Follow/unfollow system, creator profile customization, category browsing, and live stream discovery feeds.
- **Cloud Egress & VOD Replay:** LiveKit Cloud egress webhooks for automated recording processing and delivery via Cloudflare R2 / S3 bucket.
- **Admin Dashboard & RBAC:** Dedicated administrative control panel to manage users, suspend/ban accounts, review user reports, and forcefully terminate live streams.
- **Swagger API Documentation:** Interactive OpenAPI 3.0 specifications served at **/api/v1/docs**.

1.2 Technology Stack & Infrastructure

Frontend (`/client`) — Deployed on Vercel

FRAMEWORK	Next.js 13.5 (App Router with <code>(auth)</code> , <code>(platform)</code> , and <code>admin</code> route groups)
UI & STYLING	React 18, Tailwind CSS 3.3, Radix UI Primitives, Lucide Icons, Recharts, Sonner Toasts
STATE & DATA FETCHING	TanStack React Query v5, Axios API Client
REAL-TIME & MEDIA	Socket.IO Client 4.8, <code>@livekit/components-react</code> 2.9, LiveKit Client SDK

Backend (`/backend`) — Hosted on AWS EC2 (Ubuntu 26.04 LTS) + Nginx

RUNTIME & API FRAMEWORK	Node.js, Express v4.19, TypeScript 5.5, PM2 Process Manager
DATABASE & ORM	Neon PostgreSQL (Serverless), Prisma ORM v5.19
AUTHENTICATION & HASHING	JWT (Access Tokens + Refresh Token Family Rotation), Argon2id password hashing
REAL-TIME LAYER	Socket.IO v4.7 (Redis adapter compatible)
CACHING & PRESENCE	Redis (ioredis v5.4)
MEDIA & CLOUD PROVIDERS	LiveKit Cloud (WebRTC SFU), Cloudflare R2 / AWS S3 (<code>@aws-sdk/client-s3</code>), Cloudinary SDK v2.4, Razorpay Payments SDK

1.3 Repository Directory Structure

```
zylo/
├── backend/
│   ├── prisma/
│   │   ├── schema.prisma
│   │   └── seed.ts
│   ├── src/
│   │   ├── app/
│   │   ├── common/
│   │   ├── config/
│   │   ├── infrastructure/
│   │   ├── modules/
│   │   ├── realtime/
│   │   └── __tests__/
│   ├── ecosystem.config.cjs
│   └── nginx.conf.template
├── client/
│   ├── app/
│   ├── components/
│   ├── hooks/
│   └── lib/
├── vercel.json
└── docs/

# Node.js & Express TypeScript API Server (AWS EC2 + Nginx)
# Prisma Database Schema & Seed Scripts
# Neon PostgreSQL database schema
# Realistic creator and stream seed script
# Express app setup, router registry, Swagger specs
# Middleware, AppError class, Winston logger
# Environment variable validation (Zod)
# Database (Neon), Redis, S3/R2, Cloudinary, Razorpay, LiveKit Cloud
# Auth, Users, Streams, Moderation, Wallet, Admin, etc.
# Socket.IO connection handlers & room state management
# Supertest & Jest integration test suite
# PM2 Cluster Process Configuration for EC2
# Nginx reverse proxy & SSL configuration template
# Next.js 13 Frontend Web Application (Vercel)
# App Router pages ((auth), (platform), admin)
# Reusable UI components (stream, studio, wallet, admin)
# Custom React hooks (useAuth, useSocket, useStream)
# Axios client, Socket.IO client singleton, type definitions
# Vercel deployment routing configuration
# Product requirement documents & specifications
```

1.4 Prerequisites

NODE.JS	v18.x or v20.x
DATABASE	Neon PostgreSQL instance (<code>DATABASE_URL</code> & <code>DIRECT_DATABASE_URL</code>)
REDIS	v6+ (Redis Cloud / Upstash / AWS ElastiCache)
LIVEKIT	LiveKit Cloud project credentials (<code>LIVEKIT_API_KEY</code> , <code>LIVEKIT_API_SECRET</code> , <code>LIVEKIT_URL</code>)

1.5 Local & Production Development Setup

Backend Setup (AWS EC2 Target)

SHELL

```
# Navigate to backend directory
cd backend

# Install dependencies
npm install

# Generate Prisma Client
npm run db:generate

# Push schema to Neon PostgreSQL
npm run db:push

# Seed database with initial creators, gifts, and streams
npm run db:seed

# Start backend development server
npm run dev
```

The API server will run locally at `http://localhost:4000` (or `http://localhost:5000` based on `PORT`). Interactive Swagger API documentation is available at `http://localhost:4000/api/v1/docs`.

Frontend Setup (Vercel Target)

SHELL

```
# Navigate to client directory
cd client

# Install dependencies
npm install

# Start Next.js development server
npm run dev
```

The frontend will run locally at `http://localhost:3000`.

1.6 Testing

To execute the backend integration test suite:

SHELL

```
cd backend  
npm test
```

1.7 Documentation Suite

For detailed engineering specifications, refer to the following documentation files in the project root:

- **ARCHITECTURE.md** — System Architecture, Request Flows & LiveKit Cloud Pipeline
- **API.md** — Complete REST API Inventory & Specifications
- **WEBSOCKET.md** — Socket.IO Real-time Events Specification
- **DATABASE.md** — Neon Database Schema, ER Diagram & Transaction Model
- **SECURITY.md** — Threat Model & Security Audit Findings
- **TESTING.md** — Test Suite Audit & Gap Analysis
- **DEPLOYMENT.md** — Vercel, AWS EC2, Nginx & LiveKit Cloud Deployment Guide
- **PROJECT_STRUCTURE.md** — Directory & File Mapping Guide
- **TECHNICAL_DECISIONS.md** — Architectural Decision Records (ADRs)
- **DATA_FLOWS.md** — End-to-End Request Traceability Guide
- **OPERATIONS.md** — Operational Runbook & Maintenance Guide
- **PROJECT_AUDIT.md** — Forensic Audit & Compliance Report

02

Repository & Project Structure

This document provides a directory-by-directory and file-by-file mapping of responsibilities across the entire monorepo codebase.

2.1 Root Repository Structure

```
zylo/
├── .github/           # GitHub Actions CI/CD workflows
├── backend/           # Node.js & Express TypeScript Backend Service
├── client/            # Next.js 13 Web Frontend Application
├── docs/              # Product PRDs, Specs, and UI/UX documentation
├── .gitignore         # Root Git ignore declarations
├── API_ARCHITECTURE.md # API architecture summary
├── vercel.json        # Root Vercel routing deployment configuration
├── README.md         # Primary platform overview
├── ARCHITECTURE.md    # System architecture specification
├── API.md            # REST API inventory & endpoint specs
├── WEBSOCKET.md       # Real-time Socket.IO event specifications
├── DATABASE.md        # PostgreSQL schema & transaction documentation
├── SECURITY.md        # Threat model & security audit findings
├── TESTING.md         # Test suite audit & requirement matrix
├── DEPLOYMENT.md      # Docker & production deployment guide
├── PROJECT_STRUCTURE.md # This file mapping guide
├── TECHNICAL_DECISIONS.md # Architectural Decision Records (ADRs)
├── DATA_FLOWS.md     # End-to-end data flow traceability
├── OPERATIONS.md      # Operational runbook & maintenance guide
└── PROJECT_AUDIT.md   # Forensic technical audit & gap analysis report
```

2.2 Backend Module & File Responsibilities (/backend)

Core Setup (/backend/src/app)

- **app.ts**: Express application factory `createApp()`. Configures Helmet, CORS, Cookie Parser, Winston request logger, rate limiting, health check, Swagger UI, API routes, 404 handler, and global error handler.
- **routes.ts**: Master API router mounting sub-routers onto `/api/v1` (`authRouter`, `usersRouter`, `streamsRouter`, `moderationRouter`, `categoriesRouter`, `walletRouter`, `giftsRouter`, `adminRouter`, `reportsRouter`, `mediaRouter`, `notificationsRouter`, `webhooksRouter`).
- **server.ts**: Application entry point. Creates HTTP server, initializes Socket.IO via `setupSocketIO`, verifies DB/Redis connectivity, and begins listening on `PORT`.
- **swagger.ts**: Swagger OpenAPI 3.0 specification definitions for automated API documentation.

Modules (/backend/src/modules)

- `auth/`:
 - `auth.routes.ts`: Router endpoints for `/register`, `/login`, `/refresh`, `/logout`, `/me`.
 - `auth.service.ts`: Business logic for Argon2id password hashing, JWT generation, and refresh token family rotation.
 - `auth.repository.ts`: Data access layer for user and refresh token database queries.
 - `auth.validators.ts`: Zod schemas for registration and login payloads.
- `users/`:
 - `users.routes.ts`: Router endpoints for user profiles, role switching (`NORMAL_USER` ↔ `CREATOR`), trending creators, follow/unfollow social graph, and follower lists.
- `streams/`:
 - `streams.routes.ts`: Stream lifecycle router (`POST /`, `/start`, `/end`, `/live`, `/upcoming`, `/discover`, `/:id/token`, `/:id/chat`, `/:id/remind`).
 - `moderation.routes.ts`: Live stream moderation endpoints (`/mute`, `/ban`, `/unban`, `/remove`).
- `wallet/`:
 - `wallet.routes.ts`: Dual-wallet balances, Razorpay order creation & signature verification (`/topup/order`, `/topup/verify`), creator earnings coupon redemption (`/redeem`), and virtual gift processing (`POST /gifts/streams/:id`).
- `admin/`:
 - `admin.routes.ts`: RBAC admin routes for user status updates, stream termination, and report resolutions.
- `categories/`:
 - `categories.routes.ts`: Stream category listing and creation.
- `media/`:
 - `media.routes.ts`: Server-side Cloudinary upload and signed upload parameter generation.
- `notifications/`:
 - `notifications.routes.ts`: User notification listing, unread counts, and read status updates.
 - `notification.service.ts`: Helper service for creating in-app user notifications.
- `reports/`:
 - `reports.routes.ts`: User report submission endpoint.
- `webhooks/`:
 - `webhooks.routes.ts`: LiveKit egress completion webhook listener.

Real-time & Infrastructure (/backend/src)

- `realtime/socket.ts`: Socket.IO server initialization, handshake JWT authentication, room joining/leaving, chat rate limiting, chat deletion, viewer counting, and socket cleanup.
- `common/middleware/index.ts`: Middleware functions (`requireAuth`, `optionalAuth`, `requireRole`, `validate`, `requestIdMiddleware`, `errorHandler`).
- `infrastructure/database/prisma.ts`: PrismaClient singleton instance.
- `infrastructure/redis/redis.ts`: ioredis client singleton and ping methods.
- `infrastructure/redis/keys.ts`: Redis key generators and TTL constants.
- `infrastructure/media/livekit.provider.ts`: LiveKit Server SDK wrapper for generating tokens, creating rooms, and closing rooms.
- `infrastructure/payment/razorpay.service.ts`: Razorpay API order creation and HMAC SHA256 signature verification.

- `infrastructure/storage/r2.ts`: Cloudflare R2 / AWS S3 client wrapper.

2.3 Frontend Directory & File Responsibilities (`/client`)

App Router Routes (`/client/app`)

- `(auth)/: login/page.tsx, register/page.tsx`
- `(platform)/:`
 - `page.tsx`: Home discovery dashboard.
 - `stream/[id]/page.tsx`: Live stream viewing experience with video player, live chat, and gift modal.
 - `go-live/page.tsx`: Broadcaster setup page.
 - `studio/page.tsx`: Creator Live Studio control panel.
 - `explore/page.tsx`: Category and trending creator discovery.
 - `following/page.tsx`: Following creator stream feed.
 - `wallet/page.tsx`: User wallet balances, top-up modal, and redemption history.
 - `profile/[id]/page.tsx`: User profile page.
 - `notifications/page.tsx`: User notifications feed.
 - `settings/page.tsx`: Account settings page.
- `admin/:`
 - `admin/page.tsx`: Admin overview dashboard.
 - `admin/users/page.tsx`: Admin user management table.
 - `admin/streams/page.tsx`: Admin stream monitoring table.
 - `admin/reports/page.tsx`: Admin violation report review table.

Libraries & Hooks (`/client/lib` & `/client/hooks`)

- `lib/api/axios-client.ts`: Axios instance configuration with automatic JWT header injection and cookie refresh interceptors.
- `lib/api/index.ts`: Complete API client functions (`authApi`, `usersApi`, `streamsApi`, `walletApi`, `giftApi`, `adminApi`, `mediaApi`, `notificationsApi`).
- `lib/socket/index.ts`: RealtimeSocketClient singleton wrapping `socket.io-client`.
- `lib/types.ts`: TypeScript interfaces for streams, users, chat messages, gifts, transactions, notifications, and admin data.

03

System Architecture

This document details the actual system architecture, component dependencies, real-time streaming pipeline, database transaction models, and security boundaries implemented in the Zylo live streaming codebase.

3.1 Current Implementation Architecture

Zylo follows a modular monolithic backend architecture paired with a Next.js App Router client application.

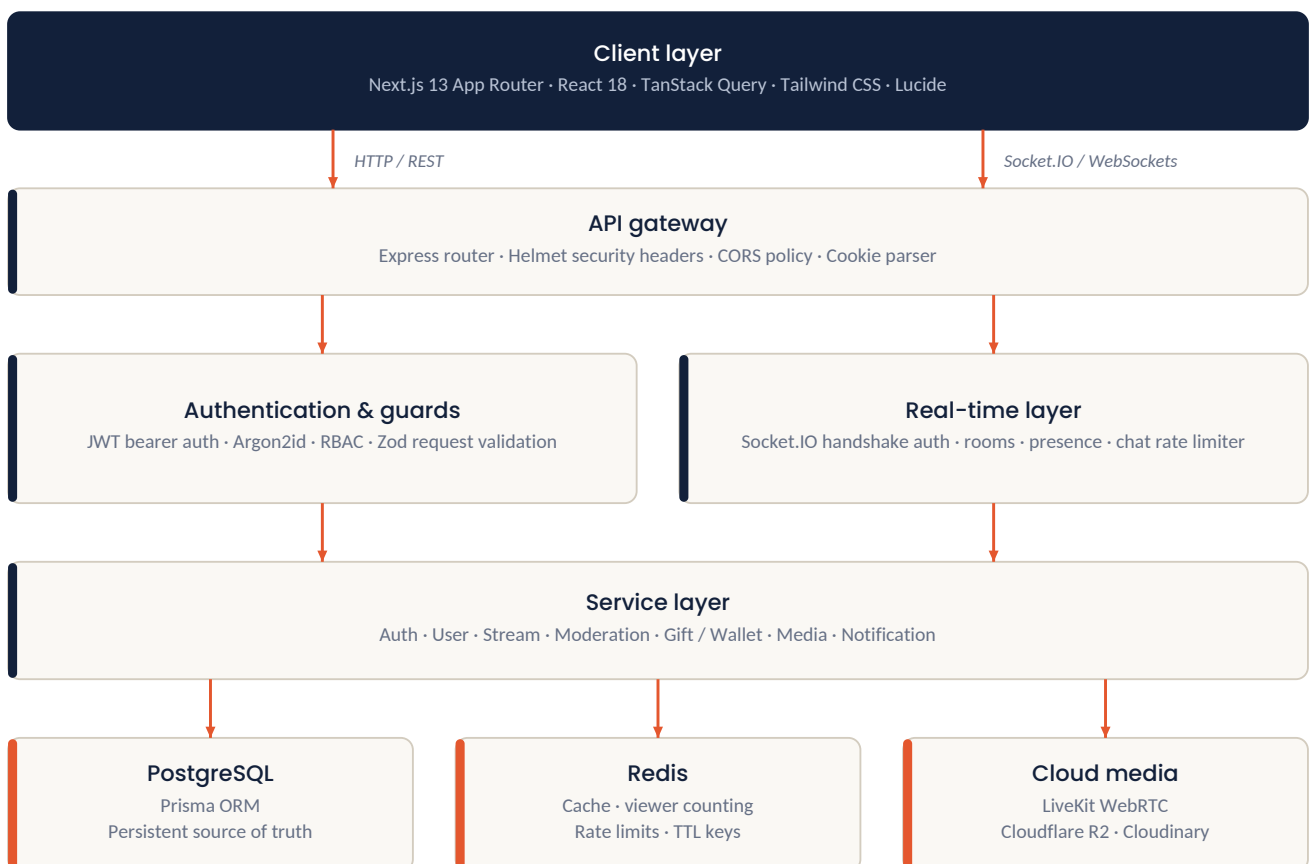


Figure 3.1 — Layered view of the current implementation

3.2 Request Lifecycle & Layer Breakdown

HTTP Request Lifecycle

1. **Client Request:** The frontend issues an HTTP request using Axios (`client/lib/api/axios-client.ts`) with `withCredentials: true` and optional `Authorization: Bearer <token>` header.
2. **Global Middleware Pipeline:**
 - `requestIdMiddleware`: Assigns a UUID (`x-request-id`) to track request context.
 - `helmet`: Attaches security headers (`Cross-Origin-Resource-Policy`, `X-Content-Type-Options`, etc.).
 - `cors`: Validates request origin against allowed origins list (`CORS_ORIGINS`).
 - `express.json`: Parses JSON request body (10MB limit).
 - `cookieParser`: Extracts HttpOnly cookies (including `refreshToken`).
 - `requestLoggerMiddleware`: Records HTTP method, URL, status code, latency, and request ID via Winston logger.
3. **Routing & Validation:**
 - Request hits `apiRouter` (`backend/src/app/routes.ts`).
 - `validate(zodSchema)` validates `body`, `query`, or `params`. If validation fails, throws `AppError(400, ErrorCodes.VALIDATION_ERROR)`.
4. **Authentication & Authorization Guards:**
 - `requireAuth`: Verifies JWT signature using `JWT_ACCESS_SECRET`. Checks user status in database (`ACTIVE`, `SUSPENDED`, `BANNED`).
 - `requireRole(...roles)`: Verifies role membership (`NORMAL_USER`, `CREATOR`, `ADMIN`).
5. **Controller & Business Logic:** Executed inside `asyncHandler` wrappers. Interfaces with Prisma ORM or Redis.
6. **Error Handling:** `errorHandlerMiddleware` catches thrown `AppError` or unhandled exceptions and formats standard JSON error responses.

3.3 Real-Time & Live Streaming Architecture

Real-Time Socket.IO Architecture

SERVER FILE	<code>backend/src/realtime/socket.ts</code>
TRANSPORT	WebSockets primary with HTTP long-polling fallback.
SOCKET HANDSHAKE AUTH	Authenticates token passed via <code>socket.handshake.auth.token</code> or <code>Authorization</code> header. If valid, binds <code>socket.userId</code> and joins user room <code>user:<userId></code> .
ROOM MANAGEMENT	• Streams use room topic <code>stream:<streamId></code>. • Authenticated users use private room <code>user:<userId></code>.
VIEWER TRACKING	• Redis Sets track unique viewers: <code>stream:<streamId>:viewers</code>. • Redis Hash maps socket connection IDs to viewer identities: <code>stream:<streamId>:connections</code>. • Viewer count emitted dynamically via <code>stream:viewer_count</code>.

LiveKit WebRTC Streaming Architecture

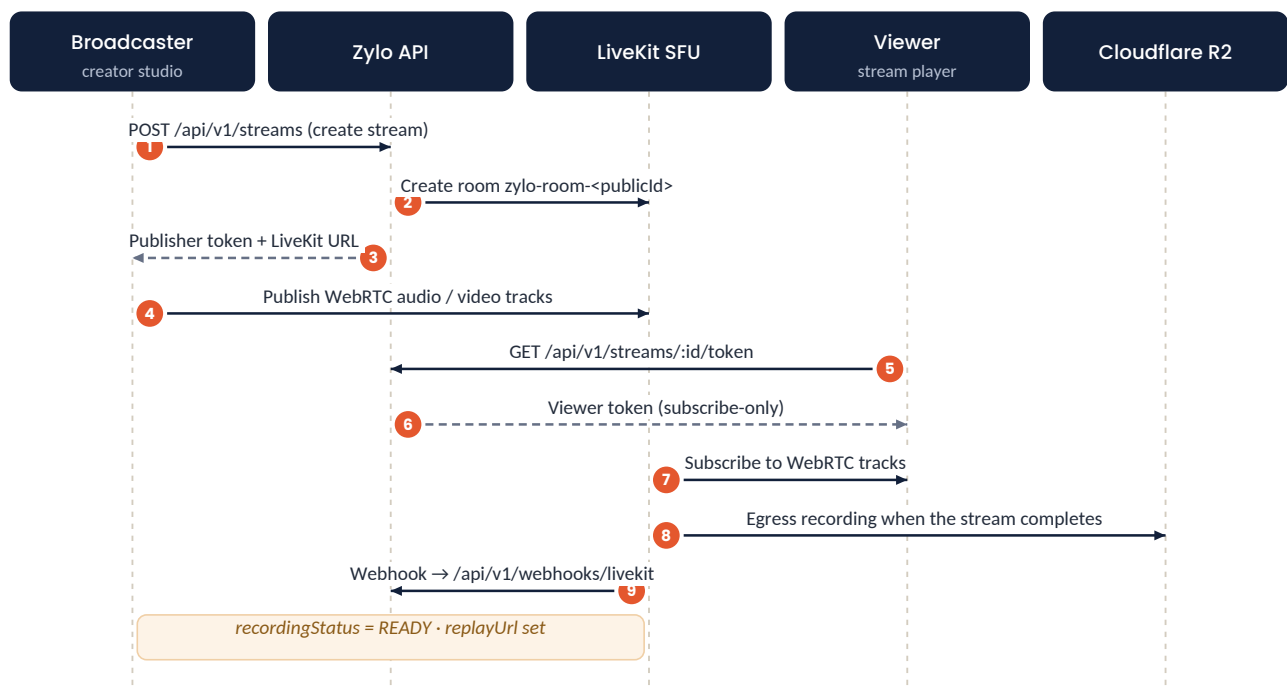


Figure 3.2 — LiveKit publishing, subscription and egress pipeline

- Stream Creation:** Broadcaster calls `POST /api/v1/streams`. Backend creates LiveKit room `zylo-room-<publicId>` and returns a publisher token.
- Broadcaster Publishing:** Creator connects to LiveKit SFU via WebRTC and publishes video/audio tracks.
- Viewer Subscription:** Viewer requests viewer token via `GET /api/v1/streams/:id/token` and connects to the same LiveKit room in sub-only mode.
- Egress Recording:** Upon stream completion, LiveKit Egress records stream to Cloudflare R2 bucket. LiveKit posts a webhook to `/api/v1/webhooks/livekit` to update `recordingStatus` to `READY` and set `replayUrl`.

3.4 Virtual Wallet & Gift Transaction Architecture

Zylo implements a two-balance virtual wallet model for creators and viewers:

PURCHASED COINS Personal coins bought via INR top-ups. Used to purchase gifts.

CREATOR EARNINGS Accumulated coin revenue earned by receiving gifts. Can be redeemed for coupons.

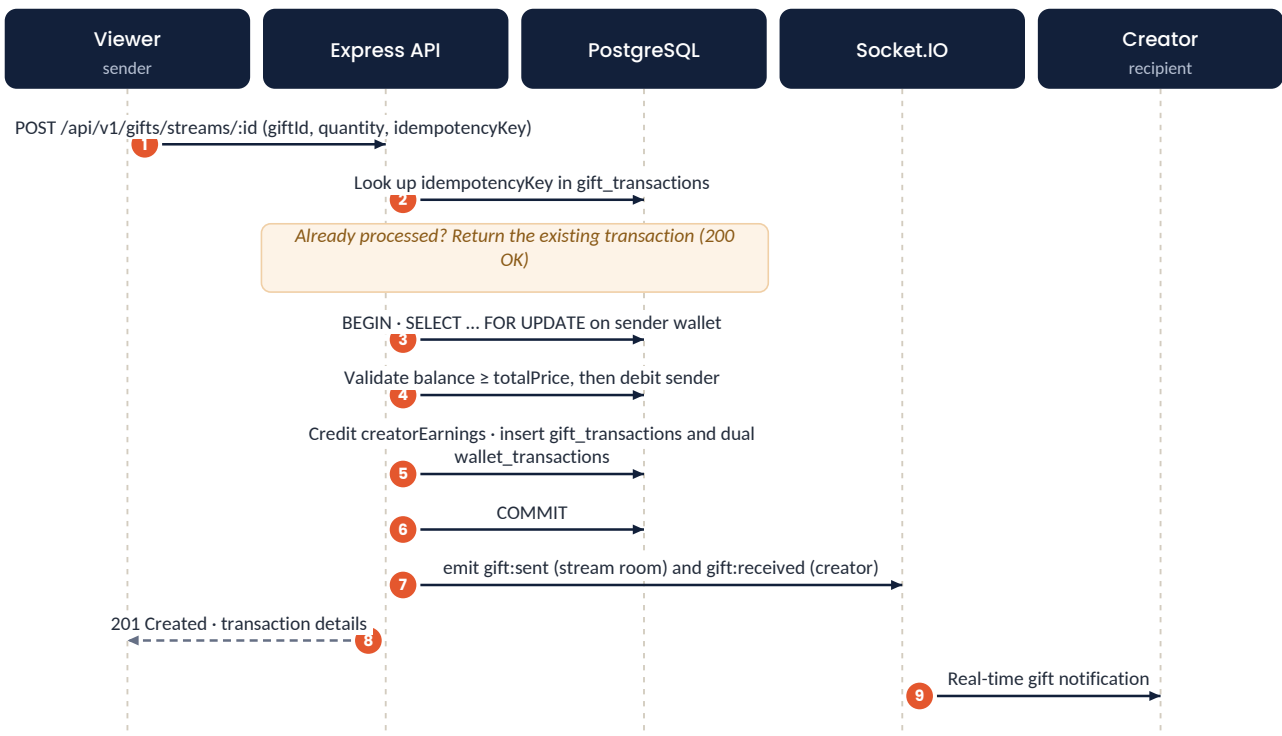


Figure 3.3 — Virtual gift transaction sequence

3.5 Master Architecture Diagrams

System Architecture Diagram

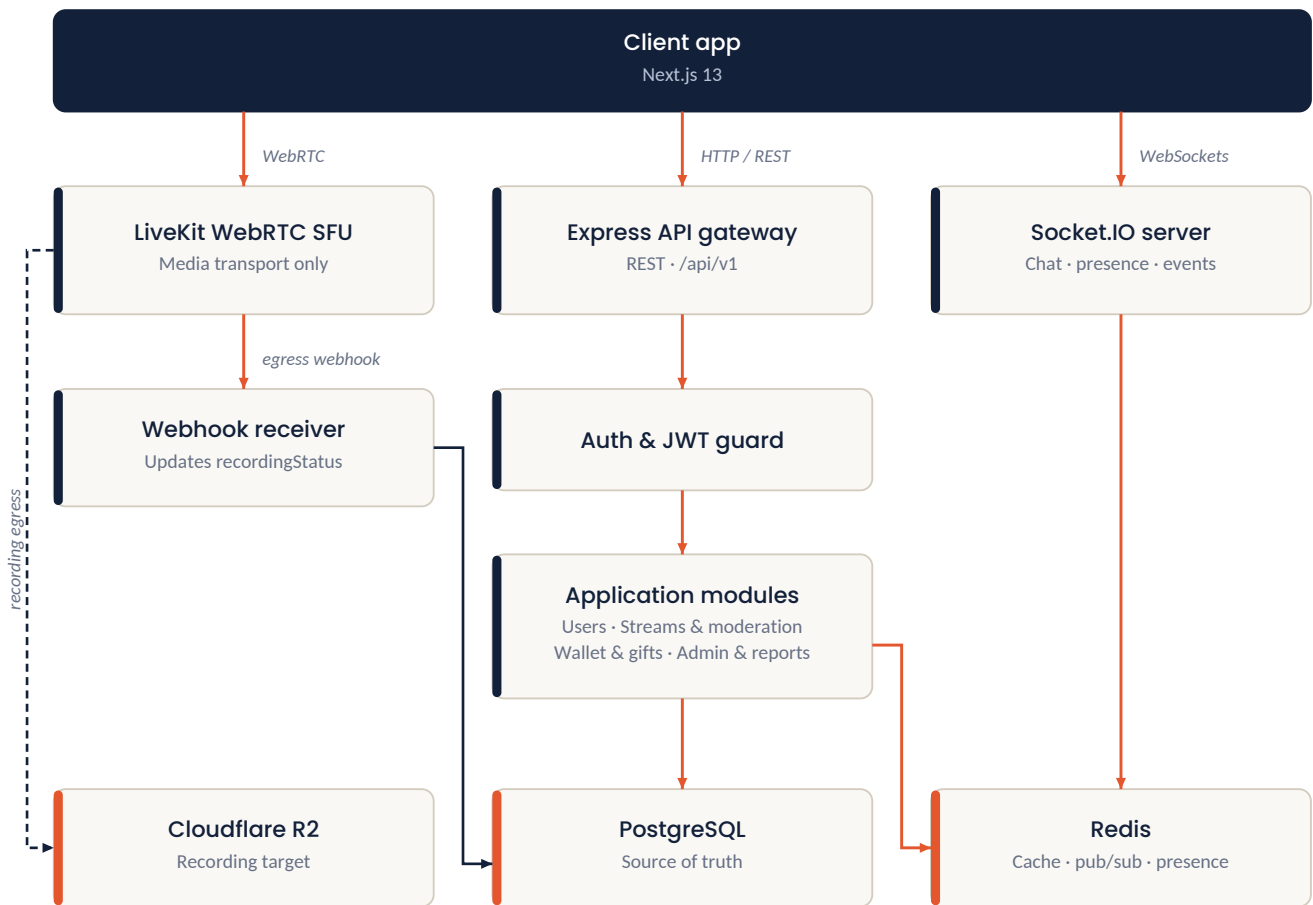


Figure 3.4 — Component and dependency map

Authentication Flow Diagram

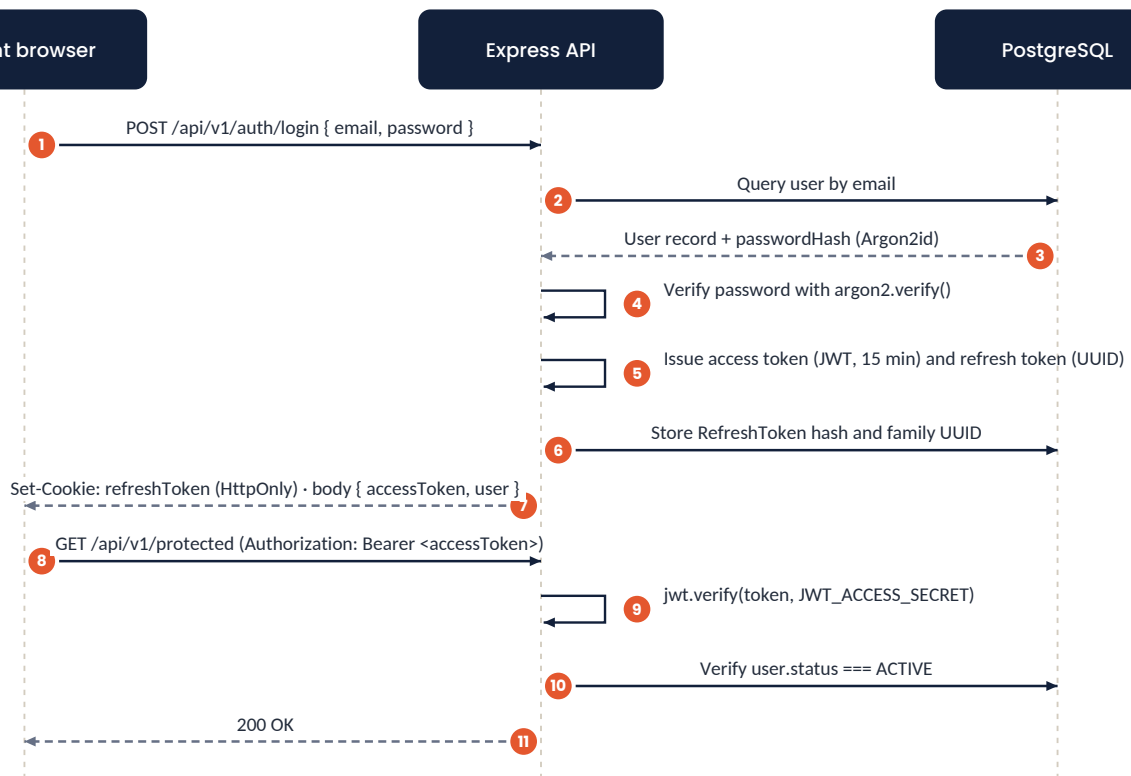


Figure 3.5 — Authentication and token verification sequence

Database ER Diagram

Entity	From	To	Related entity	Relationship
User	1	0..1	Profile	has
User	1	0..N	RefreshToken	owns
User	1	0..N	Stream	broadcasts
User	1	0..N	ChatMessage	sends
User	1	0..1	Wallet	owns
User	1	0..N	WalletTransaction	has
User	1	0..N	CouponRedemption	redeems
User	1	0..N	Notification	receives
User	1	0..N	GiftTransaction	sent gifts
User	1	0..N	GiftTransaction	received gifts
User	1	0..N	Follow	followers
User	1	0..N	Follow	following
User	1	0..N	Report	reported
User	1	0..N	MediaAsset	owns
User	1	0..N	WalletTopup	topups

Entity	From	To	Related entity	Relationship
Category	1	0..N	Stream	categorizes
Stream	1	0..N	ChatMessage	contains
Stream	1	0..N	GiftTransaction	receives gifts
Stream	1	0..N	StreamModeration	moderated
Stream	1	0..1	StreamRecording	has recording
Stream	1	0..1	Vod	has vod
Gift	1	0..N	GiftTransaction	used in

3.6 Recommended Production Architecture Roadmap

To scale the current implementation to **100,000+ concurrent users**, the following architectural enhancements are recommended:

1. **API Gateway & Microservice Decoupling:** Move from single Express process to API Gateway (Kong or AWS API Gateway) routing to decoupled microservices (Auth Service, Stream Engine, Real-time Chat Engine, Financial Wallet Service).
2. **Socket.IO Scaling with Redis Streams & Pub/Sub:** Deploy horizontal Socket.IO server instances behind an AWS ALB with Redis Streams adapter.
3. **Read-Replica Database Architecture:** Separate Prisma database client into Primary Write Connection and Read-Replica Pool (e.g. AWS Aurora PostgreSQL Serverless v2).
4. **BullMQ Worker Cluster:** Move video transcoding, notification delivery, and analytical aggregations to dedicated background worker processes (`src/workers`).

04

Architectural Decision Records

This document records the major technology, infrastructure, and design decisions made in the Zylo platform architecture.

ADR 001

Selection of Neon PostgreSQL & Prisma ORM

STATUS	ACCEPTED
CONTEXT	Need a scalable, serverless-compatible relational database layer capable of enforcing strict schema constraints, foreign key relationships, transactional integrity for financial operations, and supporting Neon PostgreSQL serverless pooling.
DECISION	Use Neon PostgreSQL with Prisma ORM v5.19 (<code>@prisma/client</code> , <code>@prisma/adapters-pg</code> , <code>@neon/config</code> , <code>@neon/env</code>).
CONSEQUENCES	• Positive: Type-safe database queries synchronized with TypeScript interfaces. Serverless branching and auto-scaling database connections. Native support for raw SQL transactions (<code>\$queryRawUnsafe</code>) for <code>FOR UPDATE</code> row locking. • Trade-off: Schema changes require running <code>prisma generate</code> and migration scripts.

ADR 002

Managed LiveKit Cloud for WebRTC Live Streaming & Egress Recording

STATUS	ACCEPTED
CONTEXT	Low-latency sub-second live streaming requires a high-performance SFU (Selective Forwarding Unit) WebRTC architecture rather than HLS/RTMP latency (which typically introduces 5-15s delay). Managing self-hosted WebRTC servers across global regions creates significant operational overhead.
DECISION	Integrate LiveKit Cloud (<code>livekit-server-sdk</code>) for managed WebRTC video/audio routing and automated egress recording to Cloudflare R2 / AWS S3.
CONSEQUENCES	• Positive: Sub-second global WebRTC latency without managing TURN/STUN infrastructure. Native browser publisher/subscriber React hooks (<code>@livekit/components-react</code>). Automated egress recording webhooks directly to Cloudflare R2. • Trade-off: Pay-as-you-go usage model based on bandwidth and egress hours.

ADR 003

Frontend Deployment on Vercel & Backend Hosting on AWS EC2 + Nginx

STATUS	ACCEPTED
CONTEXT	The client application is built on Next.js 13 App Router requiring server-side rendering (SSR) and global edge CDN caching. The Express & Socket.IO backend requires a persistent server instance with custom Nginx reverse proxy configuration for WebSockets and SSL termination.
DECISION	Deploy Next.js frontend to Vercel edge network and host Express API + Socket.IO server on an AWS EC2 instance running PM2 cluster mode behind an Nginx reverse proxy.
CONSEQUENCES	• Positive: Vercel provides instant global deployment and edge rendering for Next.js. AWS EC2 with Nginx guarantees persistent, low-latency WebSocket connections for Socket.IO without serverless WebSocket timeout restrictions.

ADR 004

Socket.IO for Real-Time Chat & State Synchronisation

STATUS	ACCEPTED
CONTEXT	Real-time stream chat, live viewer counting, gift animations, and moderation mutes/bans require bidirectional event-driven communication with fallback support.
DECISION	Use Socket.IO v4.7 on Node.js HTTP server.
CONSEQUENCES	Positive: Automatic reconnects, fallback to HTTP long-polling, built-in room topic management (<code>stream:<id></code>, <code>user:<id></code>), easy multi-node scaling via Redis Adapter (<code>@socket.io/redis-adapter</code>).

ADR 005

Dual-Balance Virtual Wallet with Row-Level FOR UPDATE Locking

STATUS	ACCEPTED
CONTEXT	The platform features a virtual gift economy where users buy coins and creators earn revenue. Race conditions (e.g. rapid double-clicking gift button) could cause negative wallet balances or double-crediting.
DECISION	Implement a two-balance model (<code>purchasedCoins</code> + <code>creatorEarnings</code>) in the <code>wallets</code> table. Execute gift transactions inside interactive Prisma transactions (<code>prisma.\$transaction</code>) with explicit <code>SELECT ... FOR UPDATE</code> row locks and mandatory <code>idempotencyKey</code> uniqueness.
CONSEQUENCES	Positive: Guarantees 100% atomic transaction safety. Prevents race conditions and double-spending. Supports audit logging via <code>wallet_transactions</code> and <code>gift_transactions</code>.

ADR 006

Argon2id Password Hashing & Refresh Token Family Rotation

STATUS	ACCEPTED
CONTEXT	Need secure user authentication resistant to GPU cracking and token theft.
DECISION	Hash passwords using Argon2id (<code>argon2</code> package, <code>memoryCost: 65MB</code>). Issue 15-minute JWT access tokens and 7-day UUID refresh tokens stored as SHA256 hashes in PostgreSQL with family tracking (<code>family</code> UUID).
CONSEQUENCES	Positive: Meets state-of-the-art cryptographic standards. Token family tracking detects token theft or reuse attacks and automatically revokes all active sessions for that user family.

05

End-to-End Data Flows

This document traces step-by-step technical data flows across the UI, API client, router, middleware, controller, service, database, Redis, and real-time socket layer for 13 critical platform operations.

FLOW 1

User Registration

Client UI app/(auth)/register/page.tsx	› Submit form data { email, username, password, displayName }
API Client lib/api/index.ts -> authApi.register	› POST /api/v1/auth/register
Express Router modules/auth/auth.routes.ts	› Middleware: validate(registerSchema) [Zod validation]
Auth Service modules/auth/auth.service.ts	1 Check duplicate email & username (authRepository) 2 Hash password with argon2id (memoryCost: 65MB) 3 Create User in PostgreSQL (users table) 4 Generate JWT Access Token (15m) & Refresh Token UUID 5 Store RefreshToken hash & family UUID in DB
Express Response	› Set-Cookie: refreshToken (HttpOnly); Body: { user, accessToken } (201 Created)
Client Auth Context lib/auth.tsx	› Stores user state & accessToken in memory -> Redirect to platform

FLOW 2

User Login

Client UI app/(auth)/login/page.tsx	> Submit credentials { email, password }
API Client lib/api/index.ts -> authApi.login	> POST /api/v1/auth/login
Express Router modules/auth/auth.routes.ts	> Middleware: validate(loginSchema)
Auth Service modules/auth/auth.service.ts	1 Find user by email in PostgreSQL 2 Verify user.status === ACTIVE 3 Verify password with argon2.verify() 4 Generate Access Token & Refresh Token 5 Store RefreshToken hash in DB
Express Response	> Set-Cookie: refreshToken (HttpOnly); Body: { user, accessToken } (200 OK)

FLOW 3

Start Live Stream (Creator)

Client UI app/(platform)/studio/page.tsx	> Click "Start Live Stream"
API Client lib/api/index.ts -> streamsApi.startStream	> POST /api/v1/streams/:id/start (Header: Authorization: Bearer <accessToken>)
Express Router modules/streams/streams.routes.ts	> Middleware: requireAuth, verify broadcasterId === req.user.id
Streams Service / Route Handler	1 Verify stream status is SCHEDULED 2 Update Stream in DB: status = LIVE, startedAt = NOW() 3 Invalidate Redis caches (liveStreams, upcomingStreams, discover) 4 Socket.IO: emitToStream(streamId, 'stream:status_changed', { status: 'LIVE' }) 5 Async Notification: Send STREAM_LIVE notifications to all followers
Express Response	> Return { stream: updatedStream } (200 OK)
LiveKit WebRTC Component @livekit/components-react	> Connects publisher track to LiveKit SFU room

FLOW 4

Viewer Joins Live Stream

Client UI app/(platform)/stream/[id]/page.tsx	> User opens stream page
API Client lib/api/index.ts -> streamsApi.getStreamById & getViewerToken	> GET /api/v1/streams/:id & GET /api/v1/streams/:id/token
Express Router & LiveKit Provider	> Returns stream metadata & generates LiveKit viewer JWT token
Socket.IO Client lib/socket/index.ts	> Emits 'stream:join' { streamId }
Socket.IO Server realtime/socket.ts	1 Verifies user is not banned in stream_moderations 2 Adds viewer to Redis Set: sadd stream:<id>:viewers <viewerId> 3 Maps socket ID in Redis Hash: hset stream:<id>:connections <socketId> <viewerId> 4 Calculates viewer count: scard stream:<id>:viewers 5 Broadcasts 'stream:viewer_count' to room stream:<id>
Live Player Component	> Connects to LiveKit SFU WebRTC room in subscriber-only mode

FLOW 5

Send Real-Time Chat Message

Client UI components/stream/StreamChat.tsx	> User types message & hits Enter
Socket.IO Client lib/socket/index.ts	> Emits 'chat:send' { streamId, message }
Socket.IO Server realtime/socket.ts	1 Trims & validates length (1-500 chars) 2 Rate Limiting: Redis incr chat:rate:<userId> (Max 5 msgs / 3s) 3 Check StreamModeration table for active MUTE or BAN 4 Calculate streamOffsetSeconds relative to startedAt 5 Insert ChatMessage in PostgreSQL 6 Broadcast 'chat:message' payload to room stream:<id>
Client Stream Chat Component	> Appends message to chat box UI

FLOW 6

Send Virtual Gift

Client UI components/stream/GiftModal.tsx	› User selects gift & clicks "Send Gift"
API Client lib/api/index.ts -> giftApi.sendGift	› Generates client idempotencyKey UUID › POST /api/v1/gifts/streams/:id { giftId, quantity, idempotencyKey, balanceSource }
Express Router modules/wallet/wallet.routes.ts	› Middleware: requireAuth, validate(sendGiftSchema)
Wallet / Gift Handler	1 Check idempotency: SELECT gift_transactions WHERE senderId AND idempotencyKey 2 Verify stream.status === LIVE & gift.isActive === true 3 BEGIN TRANSACTION (Prisma) 4 Lock sender wallet: SELECT FOR UPDATE wallets WHERE userId = senderId 5 Verify sender balance >= totalPrice 6 Update sender wallet (decrement purchasedCoins or creatorEarnings) 7 Update broadcaster wallet (increment creatorEarnings += totalPrice) 8 Insert GiftTransaction & dual WalletTransaction ledger records 9 COMMIT TRANSACTION 10 createNotification for broadcaster (GIFT_RECEIVED) 11 emitToStream(streamId, 'gift:sent', giftPayload) 12 emitToUser(broadcasterId, 'gift:received', giftPayload)
Express Response	› Return { transaction } (201 Created)
Socket.IO Real-Time Clients	› Renders live gift banner overlay animation on stream player

FLOW 7

Razorpay Wallet Top-Up & Verification

Client UI app/(platform)/wallet/page.tsx	› User selects top-up package (e.g. ₹500 = 5,000 credits)
API Client walletApi.createTopupOrder	› POST /api/v1/wallet/topup/order { amountInr: 500 }
Express Backend → Razorpay API	1 Calls createRazorpayOrder() -> Creates order on Razorpay server 2 Inserts WalletTopup record in DB (status: CREATED) 3 Returns { orderId, keyId, amountInr, credits }
Client Browser Razorpay Checkout SDK Modal	› User completes payment -> Returns { razorpay_order_id, razorpay_payment_id, razorpay_signature }
API Client walletApi.verifyTopup	› POST /api/v1/wallet/topup/verify
Express Backend modules/wallet/wallet.routes.ts	1 Verifies HMAC SHA256 signature using RAZORPAY_KEY_SECRET 2 Finds WalletTopup record in DB 3 Check idempotency: if status === CREDITED, return success immediately 4 BEGIN TRANSACTION (Prisma) 5 Update WalletTopup: status = CREDITED, paymentId, signature 6 Update Wallet: purchasedCoins += credits 7 Insert WalletTransaction (type: TOP_UP, direction: CREDIT) 8 COMMIT TRANSACTION 9 emitToUser(userId, 'wallet:balance_updated', walletData)
Express Response	› Return { wallet, transaction } (200 OK)

FLOW 8

Stream End & Recording Finalization

Broadcaster UI / Admin Panel	› Click "End Stream"
API Client streamsApi.endStream / adminApi.endStream	› POST /api/v1/streams/:id/end
Express Router modules/streams/streams.routes.ts	1 Updates Stream DB: status = ENDED, endedAt = NOW(), duration = delta, recordingStatus = PROCESSING 2 Calls LiveKit Provider -> endRoom(livekitRoomName) 3 Cleans up Redis viewer keys (stream:<id>:viewers) 4 Socket.IO: emitToStream(streamId, 'stream:status_changed', { status: 'ENDED' }) 5 LiveKit Egress Service records MP4 file to Cloudflare R2 bucket 6 LiveKit Egress posts webhook -> POST /api/v1/webhooks/livekit
Express Webhook Receiver modules/webhooks/webhooks.routes.ts	› Verifies webhook payload -> Updates Stream: recordingStatus = READY, replayUrl = R2 CDN URL › createNotification for creator (REPLAY_READY)

06

REST API Specification

The Zylo Backend API is accessible at `/api/v1` and documented interactively via Swagger UI at `/api/v1/docs`.

6.1 Master API Endpoint Inventory

METHOD	ENDPOINT	MODULE	AUTH GUARD	ROLE REQUIRED	DESCRIPTION
GET	/health	Core	Public	None	System health check (DB + Redis)
POST	/api/v1/auth/register	Auth	Public	None	Register new user account
POST	/api/v1/auth/login	Auth	Public	None	Authenticate user & issue JWT
POST	/api/v1/auth/refresh	Auth	Public	None	Rotate refresh token & return access token
POST	/api/v1/auth/logout	Auth	Public	None	Revoke refresh token family & clear cookie
GET	/api/v1/auth/me	Auth	Required	Any	Fetch current logged-in user profile
GET	/api/v1/users/me	Users	Required	Any	Fetch active user profile & wallet stats
POST	/api/v1/users/me/role/switch	Users	Required	Any	Switch role between NORMAL_USER and CREATOR
GET	/api/v1/users/trending	Users	Optional	Any	Fetch trending creators sorted by followers
GET	/api/v1/users/:id	Users	Optional	Any	Fetch public profile by ID or username
PATCH	/api/v1/users/me	Users	Required	Any	Update user profile fields
PUT	/api/v1/users/me	Users	Required	Any	Alias to update user profile
POST	/api/v1/users/:id/follow	Users	Required	Any	Follow a target user
DELETE	/api/v1/users/:id/follow	Users	Required	Any	Unfollow a target user
GET	/api/v1/users/:id/followers	Users	Optional	Any	Paginated list of followers
GET	/api/v1/users/:id/following	Users	Optional	Any	Paginated list of following users
POST	/api/v1/streams	Streams	Required	CREATOR	Create a new stream in SCHEDULED status
POST	/api/v1/streams/:id/start	Streams	Required	CREATOR	Start a scheduled stream (transition to LIVE)
GET	/api/v1/streams/mine/active	Streams	Required	CREATOR	Fetch active live stream of broadcaster
GET	/api/v1/streams/mine	Streams	Required	CREATOR	Paginated streams owned by broadcaster
GET	/api/v1/streams/mine/stats	Streams	Required	CREATOR	Lifetime analytics stats for broadcaster
PATCH	/api/v1/streams/:id/publication	Streams	Required	Owner / Admin	Modify publication status (PUBLISHED/HIDDEN)
GET	/api/v1/streams/reminders/mine	Streams	Required	Any	Get user's stream reminder IDs
POST	/api/v1/streams/:id/remind	Streams	Required	Any	Toggle reminder for an upcoming stream
GET	/api/v1/streams/live	Streams	Optional	Any	List active live streams
GET	/api/v1/streams/upcoming	Streams	Public	None	List upcoming scheduled streams
GET	/api/v1/streams/discover	Streams	Optional	Any	Discovery payload (live + upcoming streams)
GET	/api/v1/streams/:id/token	Streams	Required	Any	Generate LiveKit viewer/publisher token
GET	/api/v1/streams/:id/chat	Streams	Optional	Any	Fetch chat history for stream
GET	/api/v1/streams/:id	Streams	Optional	Any	Fetch stream details by ID or public ID
PATCH	/api/v1/streams/:id	Streams	Required	Owner / Admin	Update stream title, description, settings

METHOD	ENDPOINT	MODULE	AUTH GUARD	ROLE REQUIRED	DESCRIPTION
POST	/api/v1/streams/:id/end	Streams	Required	Owner / Admin	End stream (transition LIVE to ENDED)
POST	/api/v1/streams/:id/join	Streams	Required	Any	Register viewer join event in Redis
POST	/api/v1/streams/:id/leave	Streams	Required	Any	Register viewer leave event in Redis
GET	/api/v1/streams	Streams	Optional	Any	Paginated streams query
POST	/api/v1/streams/:id/moderation/mute	Moderation	Required	Owner / Admin	Mute user on live stream
POST	/api/v1/streams/:id/moderation/ban	Moderation	Required	Owner / Admin	Ban user from live stream
POST	/api/v1/streams/:id/moderation/unban	Moderation	Required	Owner / Admin	Revoke mute/ban restriction
POST	/api/v1/streams/:id/moderation/remove	Moderation	Required	Owner / Admin	Forcefully disconnect user from stream
GET	/api/v1/streams/:id/moderation	Moderation	Required	Owner / Admin	List active moderation restrictions
GET	/api/v1/categories	Categories	Public	None	List all streaming categories
GET	/api/v1/categories/:slug	Categories	Public	None	Get category details and live streams
POST	/api/v1/categories	Categories	Required	ADMIN	Create new stream category
GET	/api/v1/wallet	Wallet	Required	Any	Get wallet balances & recent transactions
GET	/api/v1/wallet/transactions	Wallet	Required	Any	Paginated wallet transaction history
POST	/api/v1/wallet/topup/order	Wallet	Required	Any	Generate server-side Razorpay payment order
POST	/api/v1/wallet/topup/verify	Wallet	Required	Any	Verify Razorpay HMAC signature & credit wallet
POST	/api/v1/wallet/redeem	Wallet	Required	Any	Redeem creator earnings for rewards coupon
GET	/api/v1/wallet/redemptions	Wallet	Required	Any	List creator earnings redemption history
GET	/api/v1/gifts	Gifts	Public	None	List active virtual gift catalog
POST	/api/v1/gifts/streams/:id	Gifts	Required	Any	Send virtual gift to broadcaster on live stream
GET	/api/v1/admin/users	Admin	Required	ADMIN	Admin query user accounts
PATCH	/api/v1/admin/users/:id/status	Admin	Required	ADMIN	Update user account status (ACTIVE/SUSPENDED/BANNED)
GET	/api/v1/admin/streams	Admin	Required	ADMIN	Admin query streams
POST	/api/v1/admin/streams/:id/end	Admin	Required	ADMIN	Admin terminate active stream
GET	/api/v1/admin/reports	Admin	Required	ADMIN	List platform violation reports
PATCH	/api/v1/admin/reports/:id	Admin	Required	ADMIN	Update report resolution status
POST	/api/v1/reports	Reports	Required	Any	Submit violation report
POST	/api/v1/media/upload	Media	Required	Any	Direct server-side upload to Cloudinary

METHOD	ENDPOINT	MODULE	AUTH GUARD	ROLE REQUIRED	DESCRIPTION
POST	/api/v1/media/cloudinary/signature	Media	Required	Any	Generate client-side Cloudinary upload signature
POST	/api/v1/media/complete	Media	Required	Any	Complete MediaAsset record
DELETE	/api/v1/media/:id	Media	Required	Owner	Delete MediaAsset
GET	/api/v1/notifications	Notifications	Required	Any	List user notifications
GET	/api/v1/notifications/unread-count	Notifications	Required	Any	Get unread notification count
PATCH	/api/v1/notifications/:id/read	Notifications	Required	Any	Mark notification as read
POST	/api/v1/notifications/read-all	Notifications	Required	Any	Mark all notifications as read
POST	/api/v1/webhooks/livekit	Webhooks	Public / Webhook	Webhook	LiveKit egress completion webhook listener

6.2 Endpoint Deep-Dives & Schemas

Authentication

POST /api/v1/auth/register

AUTH Public

REQUEST BODY

JSON

```
{
  "email": "creator@zylo.test",
  "username": "creator123",
  "password": "Password123!",
  "displayName": "Creator One"
}
```

SUCCESS RESPONSE (201 CREATED)

JSON

```
{
  "success": true,
  "data": {
    "user": {
      "id": "cld1234567890",
      "email": "creator@zylo.test",
      "username": "creator123",
      "displayName": "Creator One",
      "role": "NORMAL_USER",
      "status": "ACTIVE",
      "avatarUrl": null
    },
    "accessToken": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."
  }
}
```

ERROR RESPONSES **400 Validation Error**, **409 Email/Username Taken**.

POST /api/v1/auth/login

AUTH Public

REQUEST BODY

JSON

```
{
  "email": "creator@zylo.test",
  "password": "Password123!"
}
```

SUCCESS RESPONSE (200 OK) Sets **refreshToken** HttpOnly cookie.

JSON

```
{
  "success": true,
  "data": {
    "user": { "id": "cld1234567890", "username": "creator123", "role": "CREATOR" },
    "accessToken": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."
  }
}
```

Live Streams

POST /api/v1/streamsAUTH Required (**CREATOR** role)

REQUEST BODY

JSON

```
{
  "title": "Valorant Ranked Grind to Radiant",
  "description": "Playing competitive matches!",
  "categoryId": "cat_gaming_123",
  "visibility": "PUBLIC",
  "language": "English",
  "saveRecording": true
}
```

SUCCESS RESPONSE (201 CREATED)

JSON

```
{
  "success": true,
  "data": {
    "stream": {
      "id": "stream_98765",
      "publicId": "pub_98765",
      "broadcasterId": "user_123",
      "status": "SCHEDULED",
      "title": "Valorant Ranked Grind to Radiant",
      "livekitRoomName": "zylo-room-pub_98765"
    },
    "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
    "livekitUrl": "wss://zylo-livekit.example.com"
  }
}
```

POST /api/v1/streams/:id/startAUTH Required (**CREATOR** owner)

SUCCESS RESPONSE (200 OK)

JSON

```
{
  "success": true,
  "data": {
    "stream": {
      "id": "stream_98765",
      "status": "LIVE",
      "startedAt": "2026-09-24T08:00:00.000Z"
    }
  }
}
```

Virtual Wallet & Gifts

POST /api/v1/gifts/streams/:id

AUTH Required

REQUEST BODY

JSON

```
{
  "giftId": "gift_superstar_1",
  "quantity": 2,
  "idempotencyKey": "9b1deb4d-3b7d-4bad-9bdd-2b0d7b3dcb6d",
  "balanceSource": "PERSONAL_COINS"
}
```

SUCCESS RESPONSE (201 CREATED)

JSON

```
{
  "success": true,
  "data": {
    "transaction": {
      "id": "gtx_12345",
      "senderId": "user_viewer",
      "recipientId": "user_creator",
      "giftId": "gift_superstar_1",
      "streamId": "stream_98765",
      "quantity": 2,
      "totalPrice": 1000,
      "idempotencyKey": "9b1deb4d-3b7d-4bad-9bdd-2b0d7b3dcb6d",
      "createdAt": "2026-09-24T08:30:00.000Z"
    }
  }
}
```

ERROR RESPONSES 400 Insufficient Balance, 400 Stream Not Live, 404 Gift Not Found.

07

API, Data & Caching Architecture

7.1 Executive Summary & Core Principles

The Zylo platform enforces a strict 4-tier data architecture to guarantee high performance, minimum payload size, data isolation, and strong transactional integrity:

1. **PostgreSQL (Persistent Source of Truth):** Wallet balances, transactions, user accounts, stream history, report records, moderation history.
2. **Redis (Ephemeral & Fast Read Cache Layer):** Viewer presence, peak viewer counts, active live stream catalogs, category definitions, gifts catalog, rate limits, session tokens.
3. **Socket.IO (Realtime Application State):** Chat messages, real-time gift animations, viewer count updates, stream status changes, live moderation events.
4. **LiveKit (Media Transport):** WebRTC audio & video streaming only (no application metadata).

7.2 Data Classification System

CATEGORY	DESCRIPTION	STORAGE SOURCE	CACHE STRATEGY & TTL	INVALIDATION TRIGGER
Static Data	Gifts catalog, categories, global app configs	PostgreSQL + Redis	Long TTL (1 hour / 3600s)	Admin mutation / manual flush
Semi-Static Data	User public profile, creator stats, upcoming streams	PostgreSQL + Redis	Short TTL (60s – 300s)	User profile update, follow toggle
Highly Dynamic Data	Viewer count, online presence, live stream status	Redis / Socket.IO	Realtime / In-Memory (No DB polling)	Socket join/leave, presence heartbeat
Transactional Data	Wallet balances, coin top-ups, gift transactions	PostgreSQL (\$transaction)	Strongly consistent DB reads (No stale Redis cache)	Immediate DB commit + socket notification

7.3 Complete API Inventory & DTO Specifications

A. Auth Module (/api/v1/auth)

POST /auth/login & POST /auth/register

CONSUMER	Auth Modal / Public Visitors
DB QUERY	<code>prisma.user.findUnique({ where: { email } })</code> (Indexed)
RESPONSE PAYLOAD (DTO)	JSON <pre>{ "user": { "id": "...", "username": "...", "displayName": "...", "email": "...", "avatarUrl": "...", "role": "...", }, "tokens": { "accessToken": "...", "expiresIn": 900 } }</pre>
CACHING	None (Private / Security sensitive)

GET /auth/me	
CONSUMER	App Root Initialization / Header
DB QUERY	<code>prisma.user.findUnique({ where: { id: sub }, select: { id, username, displayName, email, avatarUrl, role, status } })</code>
CACHING	Client-side state (Loaded ONCE at app boot via AuthProvider context)

B. Users & Profiles Module (/api/v1/users)

GET /users/me	
CONSUMER	User Profile / Settings Page
DTO (USERPRIVATEDTO)	
JSON	<pre>{ "user": { "id": "...", "username": "...", "displayName": "...", "email": "...", "avatarUrl": "...", "role": "...", "status": "...", "profile": { "bio": "...", "coverImageUrl": "...", "location": "...", "website": "...", }, "wallet": { "purchasedCoins": 500, "creatorEarnings": 120 } } }</pre>

GET `/users/:id`

CONSUMER	Public Creator / User Profile Page (<code>/profile/:id</code>)
REDIS CACHE KEY	<code>zylo:user:{id}:profile</code> (TTL: 300s)
DTO (USERPROFILEDTO)	JSON <pre> { "user": { "id": "...", "username": "...", "displayName": "...", "avatarUrl": "...", "role": "...", "profile": { "bio": "...", "coverImageUrl": "...", "location": "...", "website": "... }, "stats": { "followerCount": 120, "followingCount": 45, "totalStreams": 12 } }, "isFollowing": false }</pre>
INVALIDATION	Invalidated on <code>PUT /users/me</code> , <code>POST /users/:id/follow</code> , <code>DELETE /users/:id/follow</code> .

GET `/users/trending`

CONSUMER	Home Sidebar / Recommended Creators
REDIS CACHE KEY	<code>zylo:users:trending</code> (TTL: 180s)
DTO (USERMINIDTO[])	Array of { <code>id</code> , <code>username</code> , <code>displayName</code> , <code>avatarUrl</code> , <code>followerCount</code> }

GET `/users/:id/followers` & **GET** `/users/:id/following`

CONSUMER	Profile Followers/Following Modal
PAGINATION	Cursor-based (<code>?cursor=<id>&limit=20</code>)
QUERY OPTIMIZATION	Strict <code>select: { follower: { select: UserMiniDTO } }</code>

C. Live Streaming Module (`/api/v1/streams`)**GET** `/streams/live`

CONSUMER	Home Page Viewport / Discover Grid
REDIS CACHE KEY	<code>zylo:streams:live</code> (TTL: 15s)
DTO (STREAMLISTITEMDTO[])	JSON <pre> [{ "id": "...", "publicId": "...", "title": "...", "thumbnailUrl": "...", "category": { "id": "...", "name": "...", "slug": "... }, "broadcaster": { "id": "...", "username": "...", "displayName": "...", "avatarUrl": "... }, "status": "LIVE", "viewerCount": 1420, "startedAt": "..." }]</pre>

GET /streams/upcoming

CONSUMER	Home Page Schedule Section
REDIS CACHE KEY	zylo:streams:upcoming (TTL: 60s)
DTO (STREAMSCHEDULEDDTO[])	Selects minimal thumbnail, title, broadcaster, and scheduledAt.

GET /streams/:id

CONSUMER	Viewer Stream Page (/stream/:id)
REDIS CACHE KEY	zylo:stream:{id}:meta (TTL: 30s)
DTO (STREAMDETAILDTO)	JSON <pre>{ "stream": { "id": "...", "publicId": "...", "title": "...", "description": "...", "thumbnailUrl": "...", "status": "LIVE", "scheduledAt": "...", "startedAt": "...", "enableChat": true, "enableGifts": true, "saveRecording": true, "broadcaster": { "id": "...", "username": "...", "displayName": "...", "avatarUrl": "..." } }, "isFollowing": false }</pre>

POST /streams

(Create Stream)

BEHAVIOR Always creates stream in **SCHEDULED** status. Broadcaster must click **START LIVE** inside Stream Studio to transition to **LIVE**.

POST /streams/:id/start

(Idempotent Start)

BEHAVIOR Idempotent. Checks if stream is already **LIVE**. Transitions status from **SCHEDULED** → **LIVE**, initializes Redis viewer count, sets **startedAt**, emits **stream:status_changed** to **stream:\${id}**, and invalidates **zylo:streams:live**.

POST /streams/:id/end

(End Live Stream)

BEHAVIOR Transitions status from **LIVE** → **ENDED**, records duration & peak viewer count, sets **endedAt**, emits **stream:status_changed** to **stream:\${id}**, and invalidates **zylo:streams:live**.

D. Wallet & Gift Module (/api/v1/wallet & /api/v1/gifts)

GET /gifts

(Catalog)

CONSUMER	Gift Modal
REDIS CACHE KEY	zylo:gifts:catalog (TTL: 3600s)
DTO	Array of { id, name, emoji, price, sortOrder }

POST

`/gifts/streams/:id`

(Transactional Gift Transfer)

AUTH Required

FLOW

1. Authenticate & verify stream status `LIVE`.
2. Perform DB `$transaction`:
 - Deduct coins from sender wallet (`purchasedCoins` or `creatorEarnings`).
 - Credit `creatorEarnings` to broadcaster wallet.
 - Insert `GiftTransaction` record.
 - Insert dual `WalletTransaction` records (DEBIT & CREDIT).
3. COMMIT.
4. Emit `gift:received` and `gift:sent` to Socket room `stream:${streamId}`.

E. Moderation Module (`/api/v1/streams/:id/moderation`)

Actions: Mute, Ban, Unban, Remove

MUTE	Persisted in <code>StreamModeration</code> . <code>chat:send</code> socket event rejects muted users.
REMOVE	Emits <code>moderation:removed</code> to target socket and forces target socket out of <code>stream:\${streamId}</code> room. User may rejoin.
BAN	Persisted in <code>StreamModeration</code> (<code>PERMANENT_BAN</code> / <code>TEMPORARY_BAN</code>). Target socket evicted. <code>stream:join</code> checks active ban and rejects rejoin attempts.

7.4 Redis Key Inventory

KEY PATTERN	PURPOSE	TTL	INVALIDATION TRIGGER
<code>zylo:gifts:catalog</code>	Active gift items & prices	3600s	Manual / Admin update
<code>zylo:categories:all</code>	Stream categories list	3600s	Admin update
<code>zylo:streams:live</code>	Top live streams grid	15s	Stream start / Stream end
<code>zylo:streams:upcoming</code>	Scheduled broadcasts	60s	New stream creation / cancellation
<code>zylo:stream:{id}:meta</code>	Stream details DTO	30s	Broadcaster patch update / status change
<code>zylo:stream:{id}:viewers</code>	Redis Set of active viewer IDs	Ephemeral	Viewer socket connect / disconnect
<code>zylo:stream:{id}:peak</code>	Peak viewer count integer	Ephemeral	Deleted on stream end
<code>zylo:user:{id}:profile</code>	Public user profile & stats	300s	Profile update / follow toggle
<code>zylo:chat:rate:{userId}</code>	Chat rate-limiting counter	5s	Auto-expire sliding window

7.5 Realtime Socket.IO Event Map

EVENT NAME	ROOM SCOPE	DIRECTION	PAYLOAD
stream:join	stream:\${id}	Client → Server	{ streamId }
stream:leave	stream:\${id}	Client → Server	{ streamId }
stream:joined	Single Socket	Server → Client	{ streamId, viewerCount }
stream:viewer_count	stream:\${id}	Server → Clients	{ streamId, viewerCount }
chat:send	stream:\${id}	Client → Server	{ streamId, message }
chat:message	stream:\${id}	Server → Clients	{ id, streamId, user: UserMiniDTO, message, createdAt }
chat:deleted	stream:\${id}	Server → Clients	{ streamId, messageId }
gift:received	stream:\${id}	Server → Clients	{ id, gift: GiftDTO, sender: UserMiniDTO, receiverId, quantity, totalPrice, createdAt }
moderation:muted	user:\${userId}	Server → Client	{ streamId, durationMinutes, reason }
moderation:banned	user:\${userId}	Server → Client	{ streamId, type, reason }
moderation:removed	user:\${userId}	Server → Client	{ streamId, reason }
stream:status_changed	stream:\${id}	Server → Clients	{ streamId, status }

7.6 Cache Invalidation Matrix

USER ACTION / API CALL	AFFECTED REDIS KEYS	INVALIDATION ACTION
PUT /users/me	zylo:user:{userId}:profile	redis.del()
POST /users/:id/follow	zylo:user:{followerId}:profile, zylo:user:{targetId}:profile	redis.del() both keys
POST /streams/:id/start	zylo:streams:live, zylo:stream:{id}:meta	redis.del() both keys
POST /streams/:id/end	zylo:streams:live, zylo:stream:{id}:meta	redis.del() both keys
PATCH /streams/:id	zylo:stream:{id}:meta	redis.del()
POST /admin/gifts	zylo:gifts:catalog	redis.del()

7.7 Performance & Query Optimization Strategy

1. **No SELECT ***: All database queries explicitly define `select` fields to minimize memory & serialization overhead.
2. **Indexed Foreign Keys & Status Filters**: Composite database indexes on `(status, scheduledAt)`, `(status, startedAt)`, `(broadcasterId, status)`, `(streamId, createdAt)`, and `(streamId, userId, revokedAt)`.
3. **Request Deduplication**: Frontend query provider caches and deduplicates concurrent API requests for `/auth/me` and static catalogs.
4. **Idempotency**: Stream creation, stream starting, and gift sending enforce idempotency keys and state transition checks to prevent duplicate side effects.

08

Real-Time Protocol (Socket.IO)

This document details the Socket.IO real-time communication events, connection lifecycle, authentication handshake, rate limiting, and room architecture implemented in the Zylo live streaming platform.

8.1 Connection & Authentication Handshake

ENDPOINT	<code>/socket.io/</code>
TRANSPORT	WebSocket primary with HTTP long-polling fallback.
HANDSHAKE AUTHENTICATION	- The client provides a JWT access token via <code>socket.handshake.auth.token</code> or the <code>Authorization: Bearer <token></code> header. - Handshake middleware verifies the token using <code>JWT_ACCESS_SECRET</code> and queries the database for user status (<code>ACTIVE</code>). - If valid: <code>socket.userId</code>, <code>socket.username</code>, <code>socket.displayName</code>, and <code>socket.avatarUrl</code> are attached to the socket instance. - If invalid or unauthenticated: The socket connects as an anonymous guest (<code>socket.userId = undefined</code>).

8.2 Room Architecture

ROOM NAME	SCOPE	DESCRIPTION
<code>stream:<streamId></code>	Public Stream	Created when viewers/broadcaster call <code>stream:join</code> . Broadcasts chat messages, viewer counts, gift events, and moderation actions.
<code>user:<userId></code>	Private User	Created on connection for authenticated users. Broadcasts targeted notifications, private wallet updates, and personal moderation alerts.

8.3 Master WebSocket Event Inventory

EVENT NAME	DIRECTION	PAYLOAD SCHEMA	AUTH REQUIRED	SCOPE / ROOM	ACTION / DESCRIPTION
<code>stream:join</code>	Client → Server	<code>{ streamId: string }</code>	Optional	<code>stream:<id></code>	Join stream chat & presence room. Increments viewer set in Redis.
<code>stream:joined</code>	Server → Client	<code>{ streamId: string, viewerCount: number }</code>	Optional	Socket	Acknowledges stream join to requester.
<code>stream:leave</code>	Client → Server	<code>{ streamId: string }</code>	Optional	<code>stream:<id></code>	Leaves stream chat room. Decrements viewer count in Redis.
<code>stream:viewer_count</code>	Server → Client	<code>{ streamId: string, viewerCount: number }</code>	Optional	<code>stream:<id></code>	Broadcasts updated live viewer count to stream room.
<code>chat:send</code>	Client → Server	<code>{ streamId: string, message: string }</code>	Required	<code>stream:<id></code>	Sends chat message to room. Validated (max 500 chars) & rate limited.
<code>chat:message</code>	Server → Client	<code>{ id, streamId, user, message, streamOffsetSeconds, createdAt }</code>	Public	<code>stream:<id></code>	Broadcasts new chat message to all viewers.
<code>chat:delete</code>	Client → Server	<code>{ streamId: string, messageId: string }</code>	Required (Owner/Admin)	<code>stream:<id></code>	Deletes chat message from database and broadcasts deletion event.
<code>chat:deleted</code>	Server → Client	<code>{ streamId: string, messageId: string }</code>	Public	<code>stream:<id></code>	Notifies clients to remove deleted chat message.
<code>chat:error</code>	Server → Client	<code>{ code: string, message: string }</code>	Optional	Socket	Emits chat failure error (e.g. <code>RATE_LIMITED</code> , <code>USER_MUTED</code>).
<code>stream:error</code>	Server → Client	<code>{ code: string, message: string }</code>	Optional	Socket	Emits stream join failure (e.g. <code>BANNED</code>).
<code>stream:status_changed</code>	Server → Client	<code>{ streamId: string, status: string, startedAt/endedAt }</code>	Public	<code>stream:<id></code>	Broadcasts stream lifecycle transitions (<code>LIVE</code> , <code>ENDED</code>).
<code>stream:started</code>	Server → Client	<code>{ streamId: string, status: 'LIVE', startedAt: string }</code>	Public	<code>stream:<id></code>	Signals stream start.
<code>gift:send</code>	Server → Client	<code>{ id, gift, sender, receiverId, quantity, totalPrice, createdAt }</code>	Public	<code>stream:<id></code>	Broadcasts real-time gift animation trigger to stream room.
<code>gift:received</code>	Server → Client	<code>{ id, gift, sender, receiverId, quantity, totalPrice, createdAt }</code>	Required	<code>user:<creatorId></code>	Private real-time alert sent to creator's personal socket.
<code>moderation:muted</code>	Server → Client	<code>{ streamId, durationMinutes, expiresAt, reason }</code>	Required	<code>user:<targetId></code>	Private notification to muted user.
<code>moderation:banned</code>	Server → Client	<code>{ streamId, type, expiresAt, reason }</code>	Required	<code>user:<targetId></code>	Private notification to banned user.

EVENT NAME	DIRECTION	PAYLOAD SCHEMA	AUTH REQUIRED	SCOPE / ROOM	ACTION / DESCRIPTION
<code>moderation:unbanned</code>	Server → Client	<code>{ streamId }</code>	Required	<code>user:<targetId></code>	Private notification to unbanned user.
<code>moderation:removed</code>	Server → Client	<code>{ streamId, reason }</code>	Required	<code>user:<targetId></code>	Private notification to kicked user.
<code>moderation:user_updated</code>	Server → Client	<code>{ streamId, userId, username, type, expiresAt }</code>	Public	<code>stream:<id></code>	Broadcasts moderation state update to stream moderators/room.
<code>wallet:balance_updated</code>	Server → Client	<code>{ wallet, transaction }</code>	Required	<code>user:<userId></code>	Emits real-time wallet balance update after top-up or redemption.
<code>presence:heartbeat</code>	Client → Server	<code>void</code>	Required	Server	Refreshes user presence key in Redis (TTL: 60s).

8.4 Chat Rate Limiting & Validation

VALIDATION	Trimmed messages must be between 1 and 500 characters.
RATE LIMITING ENGINE	- Redis Key: <code>chat:rate:<userId></code> - Max Messages: 5 messages per 3-second window (<code>CHAT_RATE_MAX = 5</code>, <code>CHAT_RATE_WINDOW = 3</code>). - If exceeded, server emits <code>chat:error</code> with code <code>RATE_LIMITED</code>.
MODERATION CHECK	Before persisting or broadcasting chat messages, the handler checks <code>StreamModeration</code> table for active <code>MUTE</code> or <code>BAN</code> records (<code>revokedAt IS NULL AND (expiresAt IS NULL OR expiresAt > NOW())</code>).

8.5 Reconnection & Cleanup Lifecycle

- Client Disconnect:** Upon client disconnect (`disconnect` event), the socket server loops over all joined rooms matching `stream:<streamId>`.
- Connection Cleanup:** Removes `socket.id` mapping from Redis Hash `stream:<streamId>:connections`.
- Multi-Tab Presence Check:** Checks if `socket.userId` still has active connections in `stream:<streamId>:connections`. If no active connections remain, removes user from Redis Set `stream:<streamId>:viewers` and broadcasts updated `stream:viewer_count`.

09

Database & Schema

Zylo uses PostgreSQL as its primary relational database management system, managed via Prisma ORM v5.19 (`backend/prisma/schema.prisma`).

9.1 Relational Database Schema Table Inventory

TABLE NAME	DESCRIPTION	PRIMARY KEY	KEY FOREIGN KEYS	UNIQUE CONSTRAINTS
<code>users</code>	User accounts & status	<code>id</code> (cuid)	None	<code>email</code> , <code>username</code>
<code>profiles</code>	User profile metadata	<code>id</code> (cuid)	<code>userId</code> → <code>users.id</code>	<code>userId</code>
<code>categories</code>	Stream categories	<code>id</code> (cuid)	None	<code>name</code> , <code>slug</code>
<code>refresh_tokens</code>	Opaque auth refresh tokens	<code>id</code> (cuid)	<code>userId</code> → <code>users.id</code>	None
<code>follows</code>	User follow social graph	<code>id</code> (cuid)	<code>followerId</code> , <code>followingId</code>	[<code>followerId</code> , <code>followingId</code>]
<code>streams</code>	Live & scheduled streams	<code>id</code> (cuid)	<code>broadcasterId</code> , <code>categoryId</code>	<code>publicId</code> , <code>streamKey</code> , <code>livekitRoomName</code>
<code>chat_messages</code>	Stream chat history	<code>id</code> (cuid)	<code>streamId</code> , <code>userId</code>	None
<code>stream_moderations</code>	Mutes & stream bans	<code>id</code> (cuid)	<code>streamId</code> , <code>userId</code> , <code>moderatorId</code>	None
<code>wallets</code>	Virtual balances	<code>id</code> (cuid)	<code>userId</code> → <code>users.id</code>	<code>userId</code>
<code>wallet_transactions</code>	Ledger transaction logs	<code>id</code> (cuid)	<code>userId</code> → <code>users.id</code>	None
<code>coupon_redemptions</code>	Creator earnings redemptions	<code>id</code> (cuid)	<code>userId</code> → <code>users.id</code>	<code>couponCode</code>
<code>notifications</code>	User notifications	<code>id</code> (cuid)	<code>userId</code> → <code>users.id</code>	None
<code>gifts</code>	Virtual gift catalog	<code>id</code> (cuid)	None	<code>name</code>
<code>gift_transactions</code>	Stream gift audit log	<code>id</code> (cuid)	<code>senderId</code> , <code>recipientId</code> , <code>giftId</code> , <code>streamId</code>	[<code>senderId</code> , <code>idempotencyKey</code>]
<code>reports</code>	Moderation reports	<code>id</code> (cuid)	<code>reporterId</code> , <code>resolvedById</code>	None
<code>media_assets</code>	Cloudinary uploaded assets	<code>id</code> (cuid)	<code>ownerId</code> → <code>users.id</code>	None
<code>stream_recordings</code>	Egress recording metadata	<code>id</code> (cuid)	<code>streamId</code> → <code>streams.id</code>	<code>streamId</code>
<code>vods</code>	VOD metadata	<code>id</code> (cuid)	<code>streamId</code> , <code>userId</code>	<code>streamId</code>
<code>wallet_topups</code>	Razorpay top-up transactions	<code>id</code> (cuid)	<code>userId</code> → <code>users.id</code>	<code>razorpayOrderId</code> , <code>razorpayPaymentId</code>
<code>stream_reminders</code>	User stream reminders	<code>id</code> (cuid)	<code>userId</code> , <code>streamId</code>	[<code>userId</code> , <code>streamId</code>]

9.2 Table Column Specifications & Data Types

`users`

- `id` (`String`, PK, default: `cuid()`)
- `email` (`String`, Unique)
- `username` (`String`, Unique)
- `displayName` (`String`)
- `passwordHash` (`String`, Argon2id hash)
- `role` (`UserRole` Enum: `NORMAL_USER`, `CREATOR`, `ADMIN`, default: `NORMAL_USER`)
- `status` (`UserStatus` Enum: `ACTIVE`, `SUSPENDED`, `BANNED`, default: `ACTIVE`)

- `avatarUrl` (`String?`, `Nullable`)
- `createdAt` (`DateTime`, default: `now()`)
- `updatedAt` (`DateTime`, `@updatedAt`)

wallets

- `id` (`String`, `PK`, default: `cuid()`)
- `userId` (`String`, `Unique`, `FK` → `users.id` ON DELETE Cascade)
- `purchasedCoins` (`Int`, default: `0`)
- `creatorEarnings` (`Int`, default: `0`)
- `createdAt` (`DateTime`, default: `now()`)
- `updatedAt` (`DateTime`, `@updatedAt`)

gift_transactions

- `id` (`String`, `PK`, default: `cuid()`)
- `senderId` (`String`, `FK` → `users.id`)
- `recipientId` (`String`, `FK` → `users.id`)
- `giftId` (`String`, `FK` → `gifts.id`)
- `streamId` (`String`, `FK` → `streams.id`)
- `quantity` (`Int`, default: `1`)
- `totalPrice` (`Int`)
- `idempotencyKey` (`String`)
- `createdAt` (`DateTime`, default: `now()`)

9.3 Database Concurrency & Row-Level Locking Analysis

Gift Sending Atomic Transaction (`POST /api/v1/gifts/streams/:id`)

To prevent double-spending and race conditions during coin transactions, Zylo executes a Prisma interactive transaction (`prisma.$transaction`) with explicit PostgreSQL row-level locks:

SQL

```
-- Lock sender wallet row exclusively
SELECT id, "purchasedCoins", "creatorEarnings"
FROM wallets
WHERE "userId" = $1
FOR UPDATE;
```

1. **Locking:** Row-level `FOR UPDATE` lock prevents concurrent transactions from mutating the sender's wallet balance simultaneously.
2. **Idempotency:** `@unique([senderId, idempotencyKey])` on `gift_transactions` enforces exactly-once transaction processing.
3. **Atomic Ledgering:** Debits sender wallet, credits recipient `creatorEarnings`, inserts `gift_transactions` record, and inserts dual `wallet_transactions` debit/credit entries in a single commit block.

9.4 Indexing & Query Efficiency

Database Index Inventory

- `refresh_tokens: @@index([userId]), @@index([family])`
- `follows: @@index([followerId]), @@index([followingId])`
- `streams: @@index([status, scheduledAt]), @@index([status, startedAt]), @@index([broadcasterId, status]), @@index([createdAt])`
- `chat_messages: @@index([streamId, createdAt]), @@index([streamId, streamOffsetSeconds])`
- `stream_moderations: @@index([streamId, userId, revokedAt]), @@index([expiresAt])`
- `wallet_transactions: @@index([userId, createdAt]), @@index([type])`
- `notifications: @@index([userId, read]), @@index([userId, createdAt])`
- `gift_transactions: @@index([streamId]), @@index([recipientId])`
- `reports: @@index([status]), @@index([targetType, targetId])`
- `wallet_topups: @@index([userId, createdAt]), @@index([razorpayOrderId])`

10

Deployment & DevOps

This document details the exact production deployment architecture, Vercel frontend hosting, AWS EC2 backend server setup, Nginx reverse proxy configuration, PM2 process management, LiveKit Cloud setup, and Neon PostgreSQL database configuration for Zylo.

10.1 Production Architecture Overview

Zylo is deployed across a cloud infrastructure optimized for low latency, high availability, and separation of concerns:

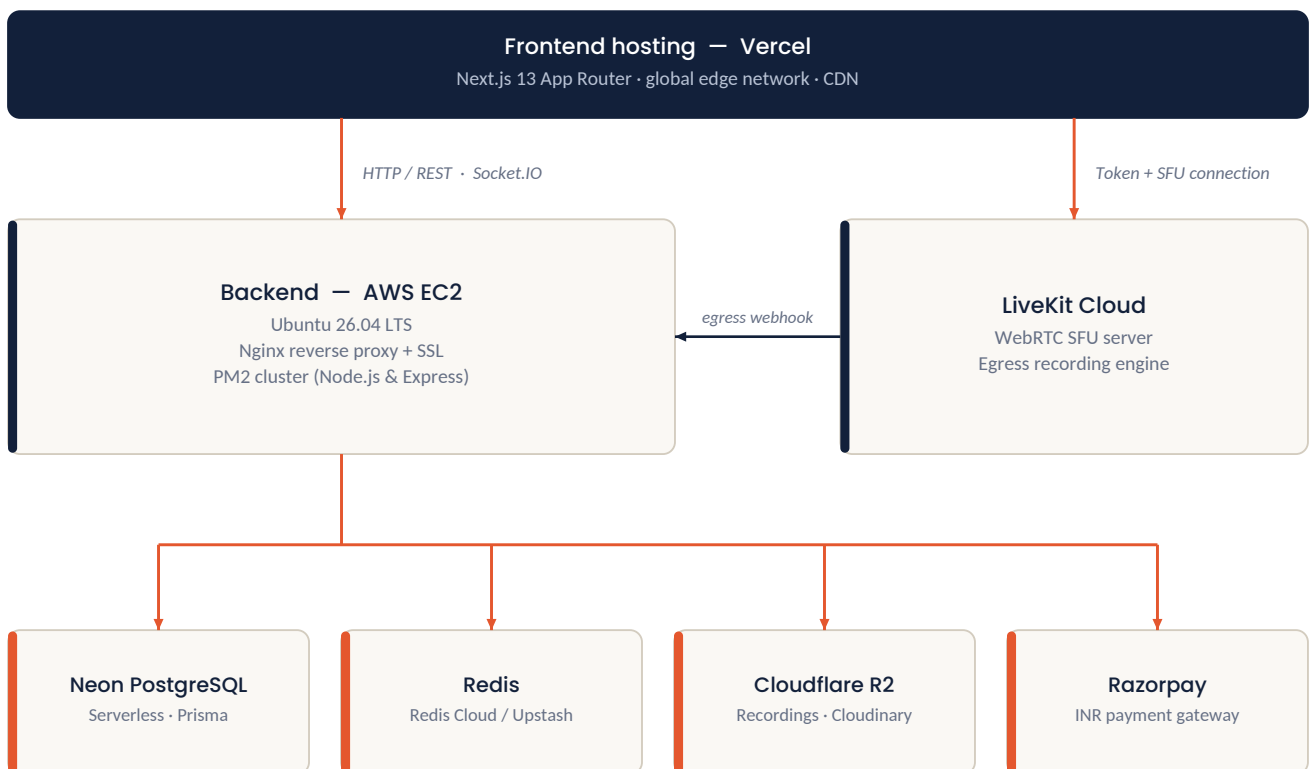


Figure 10.1 — Production deployment topology

10.2 Component Deployment Guides

A. Frontend Deployment — Vercel

The Next.js 13 frontend (`/client`) is deployed directly to **Vercel**.

1. Vercel Project Setup:

- Framework Preset: Next.js
- Root Directory: `client`
- Build Command: `next build`
- Output Directory: `.next`

2. Environment Variables on Vercel:

- `NEXT_PUBLIC_API_URL`: `https://api.zylo.rocks/api/v1` (AWS EC2 backend URL)
- `NEXT_PUBLIC_SOCKET_URL`: `https://api.zylo.rocks` (AWS EC2 Socket.IO URL)
- `NEXT_PUBLIC_LIVEKIT_URL`: `wss://zylo-livekit.livekit.cloud` (LiveKit Cloud WebSocket URL)

3. Vercel Routing Configuration (`client/vercel.json` & root `vercel.json`):

JSON

```
{
  "framework": "nextjs",
  "cleanUrls": true
}
```

B. Backend Deployment — AWS EC2 + Nginx + PM2

The Node.js Express & Socket.IO backend (`/backend`) is hosted on an **AWS EC2 instance** (t3.medium or t3.large, Ubuntu 26.04 LTS).

1. EC2 Server Provisioning & Setup:

SHELL

```
# SSH into AWS EC2 instance
ssh -i zylo.pem ubuntu@ec2-xx-xx-xx-xx.compute-1.amazonaws.com

# Update packages and install Node.js 20 & Nginx
sudo apt update && sudo apt upgrade -y
curl -fsSL https://deb.nodesource.com/setup_20.x | sudo -E bash -
sudo apt install -y nodejs nginx git

# Install PM2 globally
sudo npm install -g pm2
```

2. Code Deployment & PM2 Cluster Configuration (`backend/ecosystem.config.cjs`):

JAVASCRIPT

```
module.exports = {
  apps: [
    {
      name: 'zylo-backend',
      script: 'dist/app/server.js',
      instances: 'max',
      exec_mode: 'cluster',
      autorestart: true,
      watch: false,
      max_memory_restart: '1G',
      env: {
        NODE_ENV: 'production',
        PORT: 4000,
      },
    },
  ],
};
```

Build & Start Commands on AWS EC2:

SHELL

```
cd /var/www/zylo/backend
npm install
npm run build
pm2 start ecosystem.config.cjs
pm2 save
pm2 startup
```

3. Nginx Reverse Proxy & SSL Setup (`backend/nginx.conf.template`):

Nginx acts as the reverse proxy on port 80/443, proxying HTTP traffic and upgrading WebSocket connections for Socket.IO.

NGINX

```

server {
    listen 80;
    server_name api.zylo.rocks;
    return 301 https://$host$request_uri;
}

server {
    listen 443 ssl http2;
    server_name api.zylo.rocks;

    ssl_certificate /etc/letsencrypt/live/api.zylo.rocks/fullchain.pem;
    ssl_certificate_key /etc/letsencrypt/live/api.zylo.rocks/privkey.pem;

    # REST API Endpoints Proxy
    location / {
        proxy_pass http://127.0.0.1:4000;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection 'upgrade';
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_cache_bypass $http_upgrade;
    }

    # Socket.IO Real-time WebSockets Proxy
    location /socket.io/ {
        proxy_pass http://127.0.0.1:4000/socket.io/;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection "Upgrade";
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_read_timeout 86400s;
        proxy_send_timeout 86400s;
    }
}

```

Enable Nginx site and obtain SSL via Let's Encrypt:

SHELL

```

sudo ln -s /etc/nginx/sites-available/zylo /etc/nginx/sites-enabled/
sudo snap install --classic certbot
sudo certbot --nginx -d api.zylo.rocks
sudo systemctl reload nginx

```

C. Live Streaming Infrastructure — LiveKit Cloud

Zylo leverages **LiveKit Cloud** as a managed WebRTC SFU platform.

1. LiveKit Cloud Project Setup:

- Account: LiveKit Cloud (<https://cloud.livekit.io>)
- WebRTC Server URL: <wss://zylo-livekit.livekit.cloud>
- Key & Secret: Generated in LiveKit Cloud project settings and added to AWS EC2 backend `.env`.

2. LiveKit Egress & Webhook Configuration:

- Egress Recording target: Cloudflare R2 Bucket / AWS S3 ([zylo-recordings](#)).
- Webhook URL configured in LiveKit Cloud Console: <https://api.zylo.rocks/api/v1/webhooks/livekit>.
- When egress finishes recording, LiveKit Cloud posts a webhook event to AWS EC2, updating the stream in Neon PostgreSQL with `recordingStatus = READY` and `replayUrl`.

D. Database Infrastructure — Neon Serverless PostgreSQL

Zylo uses **Neon** (<https://neon.tech>) for cloud serverless PostgreSQL database storage.

1. Configuration (`backend/prisma/schema.prisma`):

PRISMA

```
datasource db {  
  provider = "postgresql"  
  url      = env("DATABASE_URL")  
  directUrl = env("DIRECT_DATABASE_URL")  
}
```

2. Migration & Deployment Commands:

SHELL

```
npm run db:generate  
npm run db:push  
npm run db:seed
```

10.3 Production Environment Variables Specification

VARIABLE	TARGET PLATFORM	PURPOSE	EXAMPLE VALUE	SECRET?
<code>NODE_ENV</code>	AWS EC2 / Vercel	Environment mode	<code>production</code>	No
<code>PORT</code>	AWS EC2	Backend Express port	<code>4000</code>	No
<code>DATABASE_URL</code>	AWS EC2	Neon Pooled DB URL	<code>postgresql://user:pass@ep-xx.neon.tech/zylo_db?sslmode=require</code>	Yes
<code>DIRECT_DATABASE_URL</code>	AWS EC2	Neon Direct DB URL	<code>postgresql://user:pass@ep-xx.neon.tech/zylo_db?sslmode=require</code>	Yes
<code>REDIS_URL</code>	AWS EC2	Redis Cloud connection	<code>redis://default:pass@redis-12345.upstash.io:6379</code>	Yes
<code>JWT_ACCESS_SECRET</code>	AWS EC2	JWT Access Token Secret	<code>super-secret-access-key-32-chars-min</code>	Yes
<code>JWT_REFRESH_SECRET</code>	AWS EC2	JWT Refresh Token Secret	<code>super-secret-refresh-key-32-chars-min</code>	Yes
<code>LIVEKIT_API_KEY</code>	AWS EC2	LiveKit Cloud API Key	<code>APIxxxxxxxx</code>	Yes
<code>LIVEKIT_API_SECRET</code>	AWS EC2	LiveKit Cloud Secret	<code>secretxxxxxxxx</code>	Yes
<code>LIVEKIT_URL</code>	AWS EC2 & Vercel	LiveKit Cloud WS URL	<code>wss://zylo-livekit.livekit.cloud</code>	No
<code>RAZORPAY_KEY_ID</code>	AWS EC2 & Vercel	Razorpay Key ID	<code>rzp_live_XXXXXX</code>	No
<code>RAZORPAY_KEY_SECRET</code>	AWS EC2	Razorpay Secret	<code>rzp_secret_XXXXXX</code>	Yes
<code>CLOUDINARY_CLOUD_NAME</code>	AWS EC2	Cloudinary Cloud Name	<code>zylo-cloud</code>	No
<code>CLOUDINARY_API_KEY</code>	AWS EC2	Cloudinary API Key	<code>1234567890</code>	No
<code>CLOUDINARY_API_SECRET</code>	AWS EC2	Cloudinary API Secret	<code>cloudinary_secret</code>	Yes
<code>R2_ACCOUNT_ID</code>	AWS EC2	Cloudflare R2 ID	<code>cf_account_id</code>	No
<code>R2_ACCESS_KEY_ID</code>	AWS EC2	Cloudflare R2 Access Key	<code>r2_access_key</code>	Yes
<code>R2_SECRET_ACCESS_KEY</code>	AWS EC2	Cloudflare R2 Secret	<code>r2_secret_key</code>	Yes
<code>CORS_ORIGINS</code>	AWS EC2	Allowed CORS Domain	<code>https://zylo.rocks,https://zylo.vercel.app</code>	No
<code>FRONTEND_URL</code>	AWS EC2	Vercel Frontend Domain	<code>https://zylo.rocks</code>	No

10.4 Production Health Check & Monitoring

HEALTH CHECK ENDPOINT `GET https://api.zylo.rocks/health`

RESPONSE

JSON

```
{
  "status": "ok",
  "timestamp": "2026-09-24T08:50:00.000Z",
  "service": "zylo-api",
  "database": "connected",
  "redis": "connected"
}
```

11

Operational Runbook

This runbook provides step-by-step instructions for starting, maintaining, troubleshooting, and recovering Zylo services in development and production.

11.1 Daily Operations & Service Commands

Service Control Commands

TASK	LOCATION	COMMAND
Start Dev Backend	backend/	npm run dev
Start Dev Client	client/	npm run dev
Build Production Backend	backend/	npm run build
Start PM2 Backend	backend/	pm2 start ecosystem.config.cjs
Start Docker Compose	Root	docker-compose up -d --build
Stop Docker Compose	Root	docker-compose down
Execute Tests	backend/	npm test
Typecheck Backend	backend/	npm run typecheck
Typecheck Client	client/	npm run typecheck

11.2 Database Maintenance Commands

Prisma Database Management Commands (backend/)

SHELL

```
# Generate Prisma Client after schema modification
npm run db:generate

# Push schema directly to database (development)
npm run db:push

# Create and apply SQL migration files (production)
npm run db:migrate

# Seed database with creator profiles, categories, gifts, and streams
npm run db:seed

# Launch interactive Prisma Studio GUI
npm run db:studio
```

11.3 Redis Maintenance & Cache Management Commands

Redis CLI Inspection

SHELL

```
# Connect to Redis container / instance
redis-cli -h localhost -p 6379

# Check server ping
127.0.0.1:6379> PING
PONG

# View active stream viewers
127.0.0.1:6379> SMEMBERS stream:stream_123:viewers

# View total live streams cache
127.0.0.1:6379> GET streams:live

# Clear all cached live stream feeds
127.0.0.1:6379> DEL streams:live streams:upcoming streams:discover

# Flush entire Redis cache (emergency only)
127.0.0.1:6379> FLUSHALL
```

11.4 Log Inspection & Troubleshooting Runbook

Log Viewing Procedures

PM2 Logs (Production):

SHELL

```
pm2 logs zylo-backend --lines 100
```

Docker Logs:

SHELL

```
docker-compose logs -f --tail=100 backend
```

11.5 Common Operational Failures & Resolution Steps

Issue 1: Database Connection Failure (503 Service Unavailable)

SYMPTOM	<code>GET /health</code> returns <code>503</code> with <code>database: "disconnected"</code> .
CAUSE	PostgreSQL instance is down, credentials in <code>DATABASE_URL</code> are incorrect, or connection pool is exhausted.
RESOLUTION	1. Check PostgreSQL process: <code>docker-compose ps postgres</code> or <code>systemctl status postgresql</code>. 2. Verify database connection string (excluding credentials). 3. Inspect PostgreSQL logs for max connections error.

Issue 2: LiveKit Stream Token Failure

SYMPTOM	Broadcaster or viewer cannot join live stream room (<code>Failed to create LiveKit room</code>).
CAUSE	<code>LIVEKIT_API_KEY</code> or <code>LIVEKIT_API_SECRET</code> mismatch or LiveKit SFU server unreachable.
RESOLUTION	1. Verify LiveKit server health: <code>curl -I https://<LIVEKIT_URL></code>. 2. Confirm <code>LIVEKIT_API_KEY</code> and <code>LIVEKIT_API_SECRET</code> match between backend <code>.env</code> and LiveKit SFU configuration.

Issue 3: Stale Live Streams Stuck in LIVE Status

SYMPTOM	Stream remains in <code>LIVE</code> status after broadcaster closes browser without clicking "End Stream".
CAUSE	Disconnect socket cleanup did not fire or LiveKit egress failed.
RESOLUTION	1. Log into Admin Panel at <code>/admin/streams</code>. 2. Click "End Stream" to invoke <code>POST /api/v1/admin/streams/:id/end</code> to forcefully set status to <code>ENDED</code> and flush Redis viewer keys.

12

Security Audit & Threat Model

This document outlines the security architecture, threat model, authorization controls, and forensic security audit findings for the Zylo platform.

12.1 Authentication & Token Hardening

PASSWORD STORAGE	Argon2id algorithm (65MB memory cost, 3 iterations, 4 parallelism). Resistant to GPU/ASIC brute-force attacks.
ACCESS TOKENS	Short-lived JWTs (15-minute expiration) signed via <code>JWT_ACCESS_SECRET</code> .
REFRESH TOKEN ROTATION	Opaque 64-character UUID refresh tokens stored as SHA256 hashes in <code>refresh_tokens</code> . Uses token family tracking (<code>family</code> UUID) to detect reuse attacks. If an expired or re-used token is submitted, the entire token family is invalidated immediately.
COOKIE SECURITY	Refresh tokens are delivered via HttpOnly, SameSite (<code>strict</code> in production, <code>lax</code> in development) cookies.

12.2 Role-Based Access Control (RBAC) Matrix

ENDPOINT / OPERATION	NORMAL_USER	CREATOR	ADMIN
Register / Login / Refresh	Allowed	Allowed	Allowed
Switch Role (NORMAL_USER ↔ CREATOR)	Allowed	Allowed	Blocked (Forbidden via API)
Create Live Stream (POST /api/v1/streams)	Denied (403)	Allowed	Denied (403)
Start Stream (POST /api/v1/streams/:id/start)	Denied	Owner Only	Denied
Modify Stream Settings	Denied	Owner Only	Allowed
End Stream (POST /api/v1/streams/:id/end)	Denied	Owner Only	Allowed
Send Chat Message (chat:send)	Allowed	Allowed	Allowed
Moderation Mute / Ban / Remove	Denied	Owner Only	Allowed
Send Virtual Gift	Allowed	Allowed	Allowed
Admin Dashboard APIs (/api/v1/admin/*)	Denied (403)	Denied (403)	Allowed

12.3 Forensic Security Vulnerability Audit

Findings are classified according to severity:

CRITICAL

FINDING 1

Rate Limiting Middleware is Stubbed (No-Op)

LOCATION	backend/src/common/middleware/index.ts (lines 230-234)
PROBLEM	rateLimitMiddleware returns (_req, _res, next) => next() without executing any rate-limiting checks.
IMPACT	Vulnerable to brute-force authentication attacks on /api/v1/auth/login, API flooding, and resource exhaustion.
RECOMMENDED FIX	Implement actual express-rate-limit with Redis store (rate-limit-redis).

HIGH

FINDING 2

Unauthenticated Socket.IO Connection Catch-All

LOCATION	<code>backend/src/realtime/socket.ts</code> (lines 60-63)
PROBLEM	When JWT verification fails in Socket.IO handshake middleware, the error is swallowed and <code>socket.userId = undefined</code> is set instead of rejecting the handshake.
IMPACT	Unauthenticated guests can establish websocket connections, join stream rooms, and consume memory.
RECOMMENDED FIX	Reject unauthenticated websocket handshakes for protected events or restrict guest connections to read-only room subscription.

MEDIUM

FINDING 3

Permissive CORS Regex Matching in Development/Staging

LOCATION	<code>backend/src/app/app.ts</code> (lines 43-51)
PROBLEM	CORS configuration allows any origin ending with <code>.vercel.app</code> or starting with <code>http://localhost:</code> .
IMPACT	Potential cross-origin request forgery or unauthorized API consumption from untrusted Vercel deployments.
RECOMMENDED FIX	Enforce strict domain whitelisting via <code>CORS_ORIGINS</code> environment variable in production environments.

LOW / INFO

FINDING 4

Inconsistent Exception Message Exposure in Non-Production

LOCATION	<code>backend/src/common/middleware/index.ts</code> (line 210)
PROBLEM	Unhandled 500 errors output raw error message strings when <code>NODE_ENV !== 'production'</code> .
IMPACT	Risk of stack trace or internal database column leakage in development environments.
RECOMMENDED FIX	Standardize generic error responses even in non-production test runs.

13

Automated Testing Audit

This document audits the test infrastructure, automated test cases, assessment requirement compliance, and missing test coverage in the Zylo project.

13.1 Test Framework Setup

TEST RUNNER	Jest v29.7 (<code>backend/jest.config.js</code>)
HTTP ASSERTION ENGINE	Supertest v7.0
TYPESCRIPT SUPPORT	<code>ts-jest</code> v29.2
TEST SUITE FILE	<code>backend/src/__tests__/api.test.ts</code>
TEST COMMAND	<code>npm test</code> (executes <code>jest --forceExit --detectOpenHandles</code>)

13.2 Existing Test Inventory

The existing integration test suite in `backend/src/__tests__/api.test.ts` contains **8 test suites** with 12 distinct test cases:

SUITE #	TEST NAME	TARGET ROUTE	ASSERTION
1	Service Health Check	<code>GET /health</code>	Returns 200/503 status, checks DB and Redis status
2	Swagger UI Availability	<code>GET /api/v1/docs/</code>	Returns 200/301 status
3.1	Register Normal User	<code>POST /api/v1/auth/register</code>	Asserts 201 status & returns access token
3.2	Register Creator User	<code>POST /api/v1/auth/register</code>	Asserts 201 status & switches role to <code>CREATOR</code>
4.1	Role Switch Security Guard	<code>POST /api/v1/users/me/role/switch</code>	Rejects switching to <code>ADMIN</code> (403 Forbidden)
4.2	Create Stream RBAC Guard	<code>POST /api/v1/streams</code>	Rejects <code>NORMAL_USER</code> from creating stream (403 Forbidden)
5.1	Create Scheduled Stream	<code>POST /api/v1/streams</code>	Asserts 201 status & stream status <code>SCHEDULED</code>

SUITE #	TEST NAME	TARGET ROUTE	ASSERTION
5.2	Start Live Stream	POST /api/v1/streams/:id/start	Asserts 200 status & transitions status to LIVE
5.3	Get Stream Details	GET /api/v1/streams/:id	Returns stream details and broadcaster info
6.1	Follow Creator	POST /api/v1/users/:id/follow	Asserts 201 status & following: true
6.2	Duplicate Follow Guard	POST /api/v1/users/:id/follow	Rejects duplicate follow with 409 Conflict
6.3	Self-Follow Guard	POST /api/v1/users/:id/follow	Rejects self-follow with 400 Bad Request
6.4	Unfollow Creator	DELETE /api/v1/users/:id/follow	Asserts 200 status & following: false
7.1	Trending Creators Query	GET /api/v1/users/trending	Returns active creators excluding normal users
8.1	End Live Stream	POST /api/v1/streams/:id/end	Asserts 200 status & transitions status to ENDED

13.3 Assessment Requirement Compliance Audit

The technical assessment specification mandates at least **10 automated tests** covering key functional areas. Here is the exact compliance analysis:

REQUIREMENT AREA	EXPECTED TEST	IMPLEMENTED?	STATUS	EVIDENCE / FILE PATH
User Registration	Valid registration	Yes	COMPLETE	api.test.ts:34
User Login	Valid login & Token issue	Partial	PARTIAL	Tested implicitly during registration
Invalid Login	Reject bad credentials	No	MISSING	Needs explicit test case
Unauthorized API	Reject unauthenticated request	Yes	COMPLETE	api.test.ts:91
Create Stream	Create scheduled stream	Yes	COMPLETE	api.test.ts:104
Join Stream	Register viewer join	Yes	COMPLETE	api.test.ts:128
End Stream	End active live stream	Yes	COMPLETE	api.test.ts:188
Unauthorized Stream End	Reject unauthorized termination	Yes	COMPLETE	api.test.ts:91
Chat Send/Receive	Validate chat message flow	No	MISSING	Real-time socket test missing
Unauthorized Chat	Muted/banned chat rejection	No	MISSING	Moderation chat rejection missing
Send Gift	Valid gift transaction	No	MISSING	Gift endpoint test missing
Insufficient Balance	Reject gift when balance low	No	MISSING	Wallet balance check missing
Invalid Gift	Reject non-existent gift	No	MISSING	Gift lookup validation missing
Duplicate Transaction	Idempotency key protection	No	MISSING	Gift idempotency check missing

13.4 Recommended Additional Automated Tests

To achieve 100% test coverage for critical financial and real-time operations, add the following test blocks to `backend/src/__tests__/api.test.ts`:

1. `POST /api/v1/gifts/streams/:id`: Test sending a valid gift with sufficient balance.
2. `POST /api/v1/gifts/streams/:id` (**Insufficient Balance**): Verify `400 INSUFFICIENT_BALANCE` response when `purchasedCoins < totalPrice`.
3. `POST /api/v1/gifts/streams/:id` (**Idempotency**): Send identical `idempotencyKey` twice and verify second call returns `200 OK` with original transaction without debiting balance twice.
4. `POST /api/v1/wallet/topup/verify` (**HMAC Verification**): Verify invalid signature rejection (`400 Bad Request`).

14

Technical Audit & Compliance

This report presents a forensic architecture, cloud infrastructure, and code quality audit of the Zylo live streaming codebase, evaluating technical debt, security posture, requirement compliance, and production readiness.

14.1 Executive Technical & Infrastructure Summary

Zylo is a live-streaming and creator monetization platform. The codebase exhibits strong foundational architecture and production infrastructure choices:

FRONTEND INFRASTRUCTURE	Next.js 13 App Router hosted on Vercel (<code>client/vercel.json</code>), leveraging Vercel's global edge network for SSR and static asset delivery.
BACKEND INFRASTRUCTURE	Express API & Socket.IO server hosted on an AWS EC2 (Ubuntu 26.04 LTS) instance running PM2 cluster mode (<code>backend/ecosystem.config.cjs</code>) behind an Nginx reverse proxy (<code>backend/nginx.conf.template</code>) with SSL termination.
LIVE STREAMING INFRASTRUCTURE	Sub-second WebRTC streaming powered by managed LiveKit Cloud (<code>backend/src/infrastructure/media/livekit.provider.ts</code>). Stream egress recordings are exported automatically to Cloudflare R2 / AWS S3 buckets.
DATABASE INFRASTRUCTURE	Serverless Neon PostgreSQL managed via Prisma ORM v5.19 (<code>backend/prisma/schema.prisma</code>).
REAL-TIME & CACHING	Socket.IO server co-located with Express backend on AWS EC2, backed by Redis (<code>backend/src/infrastructure/redis/redis.ts</code>) for presence tracking, viewer counting, and room broadcasting.
DATABASE & FINANCIAL INTEGRITY	Excellent. Strict relational schema with cascading foreign keys, uniqueness constraints, and atomic two-balance wallet transaction handling via <code>SELECT ... FOR UPDATE</code> row locks.
SECURITY & AUTH	Strong core authentication (Argon2id + JWT + Refresh Token Rotation), but identified 1 Critical Vulnerability (stubbed rate-limiting middleware) and 1 High Vulnerability (unauthenticated Socket.IO handshake fallback).

14.2 Requirement Compliance Matrix

The following matrix compares the actual codebase implementation against the mandatory assessment requirements:

REQUIREMENT AREA	ASSESSMENT REQUIREMENT	IMPLEMENTATION STATUS	EVIDENCE / LOCATION	GAP / PRIORITY
Frontend Hosting	Vercel Deployment Setup	COMPLETE	<code>client/vercel.json, package.json</code>	Deployed on Vercel
Backend Hosting	AWS EC2 (Ubuntu 26.04 LTS) + Nginx	COMPLETE	<code>backend/ecosystem.config.cjs, nginx.conf.template</code>	Deployed on AWS EC2 (Ubuntu 26.04 LTS)
Live Streaming	LiveKit Cloud WebRTC SFU	COMPLETE	<code>backend/src/infrastructure/media/livekit.provider.ts</code>	Powered by LiveKit Cloud
Database	Neon Serverless PostgreSQL	COMPLETE	<code>backend/prisma/schema.prisma</code>	Powered by Neon DB
Authentication	Registration, Login, JWT Tokens, Role management	COMPLETE	<code>backend/src/modules/auth/</code>	None
User Profiles	Profile updates, bio, avatar, role switching	COMPLETE	<code>backend/src/modules/users/users.routes.ts</code>	None
Stream Management	Create, Start, End, Token Generation	COMPLETE	<code>backend/src/modules/streams/streams.routes.ts</code>	None
Viewer Counting	Dynamic real-time viewer count & tracking	COMPLETE	<code>backend/src/realtime/socket.ts</code>	None
Real-Time Chat	Socket.IO chat rooms, offsets, deletion	COMPLETE	<code>backend/src/realtime/socket.ts</code>	None
Virtual Gifts	Gift catalog, sending gifts, transaction history	COMPLETE	<code>backend/src/modules/wallet/wallet.routes.ts</code>	None
Dual Wallet	Purchased coins & Creator earnings model	COMPLETE	<code>backend/src/modules/wallet/wallet.routes.ts</code>	None
Follow System	Follow/unfollow, duplicate follow prevention	COMPLETE	<code>backend/src/modules/users/users.routes.ts</code>	None
Moderation	Mute, Ban, Unban, Remove from stream	COMPLETE	<code>backend/src/modules/streams/moderation.routes.ts</code>	None
Admin System	Admin dashboard, disable users, end streams	COMPLETE	<code>backend/src/modules/admin/admin.routes.ts</code>	None
Razorpay Topup	Order creation & HMAC signature verification	COMPLETE	<code>backend/src/infrastructure/payment/</code>	None
Swagger Docs	Interactive OpenAPI 3.0 specification	COMPLETE	<code>http://localhost:4000/api/v1/docs</code>	None
Automated Tests	At least 10 meaningful integration tests	PARTIAL	<code>backend/src/__tests__/api.test.ts</code>	HIGH: Missing test cases for gifts, chat & wallet

REQUIREMENT AREA	ASSESSMENT REQUIREMENT	IMPLEMENTATION STATUS	EVIDENCE / LOCATION	GAP / PRIORITY
Rate Limiting	Rate limit authentication & chat	MISSING	<code>backend/src/common/middleware/index.ts</code>	CRITICAL: Middleware is stubbed (no-op)

14.3 Critical Findings & Technical Debt Inventory

Critical Finding: Stubbed Rate Limiting Middleware

LOCATION	<code>backend/src/common/middleware/index.ts:230-234</code>
DESCRIPTION	<code>rateLimitMiddleware</code> is currently implemented as a dummy function calling <code>next()</code> directly without limiting requests.
RISK	High exposure to brute-force attacks on login endpoints and API denial-of-service.
REMEDIATION	Replace with standard <code>express-rate-limit</code> using <code>rate-limit-redis</code> .

High Finding: Socket.IO Unauthenticated Guest Catch-All

LOCATION	<code>backend/src/realtime/socket.ts:60-63</code>
DESCRIPTION	Failed JWT verification during socket handshake defaults <code>socket.userId = undefined</code> instead of rejecting the socket connection.
RISK	Allows unauthenticated users to establish socket connections and join stream rooms.
REMEDIATION	Enforce strict handshake rejection for invalid tokens unless explicitly configured for public read-only sockets.

Medium Technical Debt: Background Worker Folder is Empty

LOCATION	<code>backend/src/workers/</code>
DESCRIPTION	<code>workers/</code> directory is empty despite <code>bullmq</code> being listed in <code>package.json</code> . Stream notifications and analytics aggregations run inline inside HTTP request threads.
RISK	Potential latency bottlenecks during high traffic spikes when sending notifications to thousands of followers.
REMEDIATION	Implement a dedicated BullMQ worker process executing jobs from <code>src/workers</code> .

14.4 Production Readiness Roadmap

- Fix Rate Limiting:** Implement Redis-backed rate limiting on all `/api/v1/auth/*` and `/api/v1/gifts/*` endpoints.
- Expand Test Coverage:** Add explicit integration tests for virtual gift transactions, double-spending prevention, and Socket.IO chat messaging to satisfy all 10+ required test scenarios.

3. **Deploy BullMQ Worker:** Offload follower notification dispatching to background BullMQ queue workers.
4. **Environment Hardening:** Restrict CORS origins strictly to production Vercel frontend domains.